

REPUBLIQUE DU CAMEROUN

PAIX - TRAVAIL - PATRIE

UNIVERSITÉ DE DSCHANG

ÉCOLE DOCTORALE



REPUBLIC OF CAMEROON

PEACE - WORK - FATHERLAND

UNIVERSITY OF DSCHANG

POST GRADUATE SCHOOL

DSCHANG SCHOOL OF SCIENCES AND TECHNOLOGY

UNITÉ DE RECHERCHE EN INFORMATIQUE FONDAMENTALE, INGÉNIERIE ET APPLICATIONS (URIFIA)

THÈME:

CONCEPTION D'UN DSL POUR LA SPÉCIFICATION DÉCLARATIVE DE PROBLÈMES DE PLANIFICATION ET LEUR RÉOLUTION PAR DES APPROCHES DE SATISFACTION DE CONTRAINTES

Mémoire présenté en vue de l'obtention du diplôme de Master en Informatique

FILIÈRE : Informatique Fondamentale
SPÉCIALISATION : Intelligence Artificielle

Par:

LOMOFOUET NDONGMO Handrey Jauress

Matricule: CM-UDS-21SCI0124

Licence en Informatique Fondamentale

Sous la direction de:

ZEKENG NDADJI Milliam Maxime

Chargé de Cours, Université de Dschang

Année académique 2025/2026

DÉCLARATION D'AUTHENTICITÉ

Je, soussigné DONFACK GIEUNANG Duval Midas, étudiant au Département de Mathématiques et Informatique, Faculté des Sciences, Université de Dschang, certifie que ce mémoire de Master intitulé "ÉVALUATION DES PERFORMANCES DE LA COMPOSITION INCRÉMENTALE DES SERVICES : UN BENCHMARK DE LA TRANSFORMÉE DE FOURIER", réalisé au sein de l'Unité de Recherche en Informatique Fondamentale, Ingénierie et Applications (URIFIA) sous la supervision du Pr. TCHOUPE TCHENDJI Maurice, est original. Il n'a jamais été publié auparavant pour l'obtention d'un quelconque diplôme.

L'étudiant

DONFACK GIEUNANG Duval Midas

.....

Le directeur de mémoire

TCHOUPE TCHENDJI Maurice

Maître de conférences, Université de Dschang

.....

Le chef de département

.....

DÉDICACES

À mes parents,

À la mémoire de SOBJIO JORELLE.

REMERCIEMENTS

Tout d'abord, je remercie infiniment *Dieu Tout-Puissant*, pour ses bénédictions, sa grâce et son infinie bonté. Sa guidance et sa protection m'ont permis de surmonter tous les obstacles et de mener à bien ce travail. À lui soit toute la gloire et l'honneur. Je tiens également à exprimer ma profonde gratitude à toutes les personnes qui ont contribué à la réalisation de ce mémoire. Sans leur soutien et leurs encouragements, ce travail n'aurait pas été possible.

- Je remercie sincèrement *Pr. TCHOUPÉ TCHENDJI Maurice*, mon encadreur académique, pour son inspiration constante et ses précieux conseils tout au long de mon parcours universitaire. Votre expertise et votre soutien indéfectible ont été des piliers essentiels dans l'élaboration de ce travail ;
- Je tiens à remercier chaleureusement *Dr. NGOUFO TAGUEU Joskel* pour son co-encadrement précieux de ce travail. Le temps qu'il a consacré à m'accompagner, ses conseils pratiques et son investissement personnel ont été des atouts majeurs pour la réussite de cette recherche. Sa disponibilité et son soutien technique m'ont permis de surmonter de nombreux défis ;
- Merci à *Dr. Adrián Puerto Aubel*, *Dr. TONLE NOUMBO Franck Bruno* pour leur soutien inestimable et leurs recommandations techniques ;
- Ma gratitude va également aux distingués membres du jury qui ont accepté d'examiner ce travail. L'attention que vous porterez à cette recherche et vos commentaires critiques constitueront un enrichissement significatif pour ma formation ;
- À tous les enseignants du département de Mathématiques-Informatique de la Faculté des Sciences de l'Université de Dschang, en particulier : *Pr. NKENLIFACK Marcellin Julius*, *Pr. TAYOU DJAMEGNI Clémentin*, *Pr. FUTE TAGNE Elie*, *Pr. BOMGNI Alain Bertrand*, *Pr. KENGNE TCHENDJI Vianney*, *Pr. SOH Mathurin*, *Dr. KENMOGNE Edith Belise*, *Dr. AZANGUEZET Benoît Martin*, *Dr. FOKOU PELAP Géraud*, *Dr. ZEKENG NDADJI Milliam Maxime*, pour la qualité de leurs enseignements et leurs précieux conseils.
- Je tiens à exprimer ma gratitude infinie envers ces deux personnes spéciales, *Mme. MEFFO Clarice*, *M. GIEUNANG Albert Roger*, pour leur soutien

constant et inconditionnel. Vous avez toujours cru en moi et m’avez donné la force et les moyens de réussir ;

- À mes frères et sœurs, *VOUGUE Arolle, LEMOFOUET Vidal, SOBJIO Elvira, WAMBA Angelo, MADONFOUET Stevia Lyncha, GIEUNANG Grace Divine*, merci pour votre soutien inconditionnel et votre amour ;
- Un grand merci aux familles *TCHOUMI, DONGMO, TEDOM, OUNDJEQUE* pour leur accompagnement et leurs conseils ;
- Je suis reconnaissant d’avoir des amis aussi précieux, comme *Leonel, Donald, Kelie, Lidelle, Williams, Genie, Ange, Gordan, Valery, Sagesse* et tout ceux que je n’ai pas pu citer qui ont partagé avec moi cette aventure académique. Votre amitié a été un véritable pilier, et je vous en suis profondément reconnaissant.
- Je tiens à exprimer ma sincère gratitude à mes aînés académiques, *Bénédicte, Séréna et Aurelle*, pour leur soutien inestimable et leurs conseils éclairés, qui ont grandement enrichi mon parcours et inspiré ma quête de connaissance.
- Que tous ceux dont les noms ne figurent pas sur cette page ne se sentent point lésés. Votre contribution est également précieuse et grandement appréciée.

TABLE DES MATIÈRES

Dédicaces	ii
Remerciements	iii
Table des Matières	v
Liste des acronymes	viii
Liste des tableaux	ix
Table des figures	x
Résumé	xi
Abstract	xiii
Introduction	1
Contexte	1
Problématique	2
Objectif	3
Structure du document	3
Chapitre 1 ► Généralités sur la composition des services	5
I.1 - Introduction	5
I.2 - Calcul Orienté Services	7
I.2.1 - Architecture Orientée Services	7
I.2.1.1 - Définition	8
I.2.1.2 - Propriétés	8
I.2.1.3 - Modèle SOA	9
I.2.2 - Évolution des microservices	10
I.2.2.1 - Définition	10
I.2.2.2 - Différence entre SOA et microservices	10
I.2.2.3 - Avantages et défis des microservices	11
I.3 - Composition des Services	12
I.3.1 - Approches de composition de services	13
I.3.1.1 - Composition statique	13
I.3.1.2 - Composition dynamique	14
I.3.2 - Vers la composition incrémentale	15

I.3.2.1 - Motivations	15
I.3.2.2 - Principes	16
I.3.2.3 - Grammaires Attribuées Gardées (GAG)	17
I.4 - Distribution des services	20
I.4.1 - Le cloud computing	21
I.4.2 - La virtualisation	24
I.4.3 - Virtualisation par conteneurs	26
I.4.4 - Comparaison entre conteneur Docker et VMs	28
I.5 - Orchestration des conteneurs	30
I.6 - Généralités sur la transformée de Fourier (TF)	32
I.6.1 - Notions fondamentales sur les signaux	32
I.6.2 - Traitement des signaux numériques	33
I.6.2.1 - Transformée de Fourier Discrète	33
I.6.2.2 - Transformée de Fourier Rapide (FFT)	34
I.6.3 - Quelques domaines d'application de la TF	37
I.7 - Conclusion	37
Chapitre II ► État de l'art sur la composition incrémentale des services	39
II.1 - Introduction	39
II.2 - Description des Services Web	40
II.3 - Taxonomie sur la composition de services	41
II.4 - Calcul incrémental et exécution paresseuse	44
II.4.1 - Les GAG pour la collaboration distribuée	45
II.4.2 - Processus d'entreprises distribués et disponibles via GAG et Publish/Subscribe	46
II.4.3 - Vers une exécution incrémentale dans les architectures orien- tées services	48
II.4.3.1 - Architecture orientée services basée sur les calculs incrémentaux	48
II.4.3.2 - Moteur de composition incrémentale	49
II.5 - Conclusion	51
Chapitre III ► Un benchmark d'évaluation de la composition incrémen- tale des services	52
III.1 - Introduction	52
III.2 - Mise en place de la solution	53
III.2.1 - Approche méthodologique	54
III.2.2 - Architecture du prototype	57

III.2.3 - Processus de déploiement de l'algorithme	58
III.3 - Conditions expérimentales	61
III.3.1 - Environnement d'expérimentation	61
III.3.2 - Paramètres de test	62
III.4 - Résultats et observations	63
III.4.1 - Exécution distribuée	63
III.4.2 - Comparaison approche incrémentale et non incrémentale . .	65
III.5 - Conclusion	68
Conclusion générale	69
Problématique étudiée et choix méthodologiques	69
Analyse critique des résultats obtenus	70
Quelques perspectives	70
Bibliographie	71

LISTE DES ACRONYMES

AWS	Amazon Web Services ;
BPEL	Business Process Execution Language ;
BPEL4WS	Business Process Execution Language for Web Services ;
BPM	Business Process Management ;
CFG	Context-Free Grammar ;
DPR	Diviser Pour Régner ;
EJB	Entreprise Java Beans ;
FFT	Fast Fourier Transform ;
GAG	Guarded Attribute Grammars ;
IA	Intelligence Artificielle ;
IBM	International Business Machines ;
JSON	JavaScript Object Notation ;
NIST	National Institute of Standards and Technology ;
PDDL	Planning Domain Definition Language ;
REST	Representational State Transfer ;
RPC	Remote Procedure Call ;
SMA	Système Multi Agent ;
SOAP	Simple Object Access Protocol ;
SOA	Service Oriented Architecture ;
TFD	Transformée de Fourier Inverse ;
TF	Transformée de Fourier ;
UDDI	Universal Description, Discovery, and Integration ;
UML	Unified Modeling Language ;
URI	Uniform Resource Identifier ;
VM	Virtual Machine ;
VMM	Virtual Machine Monitor ;
XML	eXtensible Markup Language ;
WSDL	Web Services Description Language.

LISTE DES TABLEAUX

I - Comparaison entre SOA et Microservices	11
II - Comparaison des approches de composition des services	15
III - Comparaison entre machines virtuelles et conteneurs Docker	29
IV - Comparaison entre l'approche non incrémentale et l'approche incrémentale	68

TABLE DES FIGURES

1 - Architecture Orientée Service [27, 38]	10
2 - Composition des services web [43]	13
3 - Orchestration de services web [43]	14
4 - Chorégraphie des services web [43]	14
5 - Niveaux de services dans le cloud [41]	23
7 - Architecture de Docker [14]	27
8 - Comparaison de l'architecture d'une machine virtuelle et d'un conteneur Docker [14]	28
9 - Orchestrateur des conteneurs	31
10 - Signal continue	32
11 - Signal discret	33
12 - Taxonomie des approches de composition des services	44
13 - Espace de travail actif d'un clinicien [4]	46
14 - Architecture du prototype [48]	58
15 - Temps d'exécution moyen en fonction de la taille d'entrée pour diffé- rentes configurations de parallélisme	63
16 - Algorithme non incrémental	66
17 - Algorithme incrémental	66

RÉSUMÉ

Dans un contexte technologique en perpétuelle évolution, les architectures orientées services (SOA) et les microservices se sont imposés comme des paradigmes incontournables pour concevoir des systèmes logiciels modulaires, flexibles et distribués. Ces architectures permettent de décomposer les fonctionnalités d’une application en services autonomes, facilitant ainsi la réutilisabilité, l’évolutivité et l’adaptabilité. Cependant, la manière dont ces services sont composés reste un enjeu central pour garantir performance, robustesse et réactivité, surtout dans les environnements dynamiques tels que le cloud computing ou les systèmes orientés données. Les approches classiques de composition – *statique ou dynamique* – montrent leurs limites face aux contraintes modernes de scalabilité, de résilience et de traitement paresseux. C’est dans ce cadre que s’inscrit la composition incrémentale des services, une approche émergente orientée données, où l’exécution d’un processus s’effectue de manière progressive, guidée par la disponibilité des informations calculées et attendues par les services du processus. Elle s’appuie notamment sur le modèle des Grammaires Attribuées Gardées (GAG), qui permet de représenter la logique de composition de services sous forme de règles pilotées par des dépendances explicites entre les données. La particularité de la composition incrémentale de services est qu’elle supporte la conteneurisation, qui est largement utilisée dans les architectures cloud et par les entreprises mainstream telles que Google, Amazon et IBM. Ceci la démarque des approches traditionnelles basées sur des modèles ad hoc ne supportant pas la conteneurisation. Afin d’évaluer rigoureusement cette approche, nous avons proposé un benchmark dédié à la transformée de Fourier incrémentale, déployé sur un cluster Kubernetes d’une machine physique. Les résultats expérimentaux montrent que la composition incrémentale offre une meilleure adaptabilité à l’environnement d’exécution et un parallélisme naturel qui réduit significativement le temps d’exécution. Bien plus, elle permet de proposer des implémentations plus robustes qui traitent de plus grandes quantités de données tout en offrant une meilleure résilience aux pannes. Nos résultats se généralisent à tout autre algorithme suivant le schéma DPR (Diviser pour régner).

Mots clés : Architecture Orientée services, Grammaires Attribuées Gardées, Services paresseux, composition incrementale, benchmark, transformée de Fourier, Kubernetes, cloud computing.

**EVALUATING THE PERFORMANCE OF
INCREMENTAL SERVICE
COMPOSITION : A BENCHMARK OF
THE FOURIER TRANSFORM**

ABSTRACT

In a constantly evolving technological landscape, Service-Oriented Architectures (SOA) and microservices have become essential paradigms for designing modular, flexible, and distributed software systems. These architectures enable the decomposition of application functionalities into autonomous services, thereby promoting reusability, scalability, and adaptability. However, the way these services are composed remains a critical challenge for ensuring performance, robustness, and responsiveness—particularly in dynamic environments such as cloud computing or data-driven systems. Traditional composition approaches —*whether static or dynamic*— show limitations when confronted with modern requirements for scalability, resilience, and lazy evaluation. This is where incremental service composition comes into play : a data-driven emerging approach in which the execution of a process unfolds progressively, guided by the availability of computed and expected data between services. It relies on the Guarded Attribute Grammars (GAG) model, which represents service composition logic through rules driven by explicit data dependencies. A key advantage of incremental composition is its native compatibility with containerization, a widely adopted practice in cloud architectures by major companies such as Google, Amazon, and IBM. This sets it apart from traditional approaches based on ad hoc models that do not support container-based deployments. To rigorously evaluate this approach, we designed a benchmark based on incremental Fourier Transform, deployed on a Kubernetes cluster running on a physical machine. Experimental results demonstrate that incremental composition provides better adaptability to the execution environment, naturally supports parallelism to significantly reduce runtime, and enables more robust implementations capable of handling larger data volumes while offering increased fault tolerance. Our findings generalize to any algorithm that follows the Divide and Conquer paradigm.

Keywords : Service Oriented Architecture (SOA), Guarded Attribute Grammars (GAG), Lazy services, incremental composition, benchmark, Fourier Transform, Kubernetes, cloud computing.

INTRODUCTION

SOMMAIRE

Contexte	1
Problématique	2
Objectif	3
Structure du document	3

Contexte

Le paysage logiciel a considérablement évolué, passant d'applications monolithiques à une approche de plus en plus modulaire. Les applications modernes sont construites à partir de composants indépendants et agnostiques, appelés conteneurs [34]. Contrairement aux composants traditionnels, les conteneurs encapsulent tout le code source ainsi que toutes les bibliothèques nécessaires à leur exécution. Cela permet un déploiement simplifié dans tous les environnements, à la fois locaux et cloud. Le déploiement d'applications conteneurisées se fait au travers du processus d'orchestration. L'orchestration est le processus automatisé de planification et de gestion des déploiements de conteneurs dans un cluster de machines. Il tient compte de diverses exigences de service, telles que la mémoire, la puissance de calcul (nombre de processeurs), les répliques (copies) souhaitées ou même l'emplacement physique de l'instance de conteneur. L'orchestration a été largement adoptée par les entreprises technologiques mainstream telles que Google, AWS et IBM[42, 3]. En plus de tirer parti de l'orchestration de conteneurs pour leurs propres applications, elles proposent également à d'autres entreprises des services d'orchestration basés sur le cloud. Un orchestrateur particulier, connu sous le nom de Kubernetes[6], s'est largement répandu. Développé conjointement par Google et la Fondation Linux en 2014, Kubernetes utilise le concept de pod comme unité fondamentale de calcul au sein d'un cluster. Un pod peut héberger plusieurs conte-

neurs, et un service est déployé en répliquant les pods à travers le cluster, chacun contenant les conteneurs nécessaires à l'exécution du service. Cela présente plusieurs avantages. Fiabilité accrue : en configurant plus de répliques dans les spécifications de déploiement de Kubernetes, un service devient plus résilient aux pannes. Déploiement continu simplifié : les mises à jour et les retours en arrière pour les services conteneurisés sont optimisés de façon à avoir des temps d'arrêt de services nuls.

Une innovation récente de l'équipe Fuchsia en 2024 [47] introduit le concept de composition incrémentale des services au sein de Kubernetes. Cela permet de définir de nouveaux services en combinant paresseusement des services existants déjà déployés dans le cluster. Ces services composites sont essentiellement des ensembles de coroutines (processus conçus pour le travail parallèle et coopératif). Chaque coroutine peut être soit un service ordinaire (service Kubernetes déjà déployé ou programme Java compilé), soit un autre service composite lui-même. Le principal avantage d'une telle composition de services réside dans sa nature incrémentale. Les services composites traitent leurs entrées de manière paresseuse et fournissent leurs résultats de manière incrémentale. Cette efficacité est obtenue en tirant parti de la terminaison individuelle des coroutines. Chaque coroutine fonctionne de manière indépendante et dirigée par les données. Lorsque certaines de ses données d'entrée deviennent disponibles, la coroutine s'active, produisant les sorties en fonction des entrées disponibles. Le service composite peut alors s'appuyer sur les sorties produites pour renvoyer certaines de ses propres sorties. L'équipe Fuchsia a utilisé ce concept de calcul incrémental pour déployer un service composite de tri sur un cluster virtuel d'un ordinateur portable moyen de gamme. Les résultats ont montré que la composition incrémentale de services sur Kubernetes présente de bonnes performances en matière de distribution, de parallélisme et de robustesse. À cette fin, l'équipe a développé un langage dédié à la composition de services sur Kubernetes. Le langage développé combine la syntaxe Yaml au langage de programmation Java. Les résultats présentés par l'équipe Fuchsia sont très prometteurs. Ils offrent la possibilité de composer et de distribuer des services de la même manière que l'on écrirait un programme dans un langage générique, avec la différence que les procédures sont remplacées par des services.

Problématique

Dans les architectures logicielles modernes, la composition de services permet de construire des processus complexes à partir d'unités logicielles autonomes et

réutilisables. Les approches classiques de composition, qu’elles soient statiques ou dynamiques, reposent généralement sur une orchestration centralisée et une définition globale du processus, ce qui limite leur capacité à s’adapter à des environnements fortement distribués, évolutifs et pilotés par les données.

Or, les infrastructures cloud-native actuelles — *où les services sont conteneurisés, distribués sur plusieurs nœuds, et déclenchés en fonction d’événements asynchrones* — requièrent des modèles de composition plus flexibles, capables de réagir localement aux changements de contexte, de s’exécuter partiellement et de manière paresseuse, et de tirer parti du parallélisme inhérent à certaines applications.

La composition incrémentale des services, fondée sur les grammaires attribuées gardées (GAG), constitue une réponse prometteuse à ces enjeux. Cependant, cette approche reste encore peu étudiée d’un point de vue expérimental : ses performances, sa robustesse, et sa capacité à exploiter efficacement les ressources cloud-native doivent être évaluées concrètement à travers des cas d’usage pertinents.

C’est dans cette perspective que s’inscrit la problématique de notre travail qui est de savoir comment concevoir, déployer et mesurer l’efficacité d’un moteur de composition incrémentale dans un environnement distribué ?

Objectif

L’objectif de ce travail consiste à mettre en place et valider un benchmark reproductible de la transformée de Fourier, afin de comparer de manière rigoureuse les performances des compositions de services incrémentale et non incrémentale dans un environnement distribué.

Structure du document

La suite de ce document se compose de trois chapitres et d’une conclusion générale, qui abordent les points suivants de manière approfondies :

Chapitre I : Généralités sur la composition des services : Dans ce chapitre, nous présenterons les aspects importants de l’architecture orientée services et des micro-services puis nous présenterons une vue approfondie de l’algorithme de transformée de Fourier.

Chapitre II : État de l’art sur la composition incrémentale des services : Nous y explorerons les limites des approches classiques de composition, puis nous introduisons les principes de la composition incrémentale. Une attention particulière est portée au modèle formel des Grammaires Attribuées Gardées (GAG), ainsi qu’aux travaux de recherche qui ont contribué à son développement.

Chapitre III : Contribution au benchmark d’évaluation de la composition incrémentale : Nous présenterons spécifiquement la démarche entreprise depuis l’approche GAG de l’algorithme jusqu’au déploiement sur un cluster Kubernetes et l’analyse des résultats.

Conclusion générale : Le bilan du travail se fera par le rappel de la problématique, une analyse critique des résultats obtenus, puis la présentation des perspectives.

I

CHAPITRE

GÉNÉRALITÉS SUR LA COMPOSITION DES SERVICES

SOMMAIRE

I.1 - Introduction	5
I.2 - Calcul Orienté Services	7
I.3 - Composition des Services	12
I.4 - Distribution des services	20
I.5 - Orchestration des conteneurs	30
I.6 - Généralités sur la transformée de Fourier (TF)	32
I.7 - Conclusion	37

I.1. Introduction

Dans un monde où les systèmes informatiques deviennent de plus en plus complexes et interconnectés, les architectures orientées services (SOA) [38] et les paradigmes de microservices se sont imposés comme des approches incontournables pour assurer la modularité, la réutilisabilité et la scalabilité des applications. Ces architectures permettent de décomposer les fonctionnalités en unités indépendantes, appelées services, qui peuvent être combinées dynamiquement pour répondre à des besoins variés.

Le calcul orienté service occupe aujourd'hui une place importante dans les approches modernes de structuration des systèmes logiciels complexes. Bien que la notion de service ne soit pas nouvelle, elle reste difficile à cerner dans une définition unique, tant elle s'exprime dans plusieurs domaines. *Par exemple en électronique domestique, un lecteur DVD fournit un service de lecture ; en télécommunications, un opérateur offre des services de messagerie ou de connectivité mettant en liaison*

deux utilisateurs ; en informatique, un système d'exploitation expose des services d'abstraction matérielle, tandis qu'un fournisseur cloud met à disposition des services de stockage, de calcul ou d'intelligence artificielle.

Dans les domaine du génie logiciel et du cloud computing, le concept de service est un paradigme en constante évolution. Il s'inscrit dans une trajectoire historique qui a vu émerger successivement le paradigme procédural où la structuration se faisait par fonctionnalité, le modèle objet avec l'encapsulation des données et des traitements associés, puis les composants logiciels, qui exposent des points d'entrée normalisés pour faciliter la réutilisation. L'orientation service constitue l'aboutissement naturel de cette progression : elle vise à exposer des fonctionnalités accessibles via des interfaces explicites, tout en dissimulant les détails d'implémentation. Ces fonctionnalités sont consommées via des échanges de messages, souvent standardisés, dans un cadre *loosely coupled* [38] (faiblement couplé).

L'une des avancées majeures liées à cette orientation se situe dans la désynchronisation entre la structure logicielle interne et l'utilisation externe : un service peut être distribué, répliqué, mis à l'échelle ou remplacé indépendamment des composants qui le consomment. À l'opposé des objets distribués, dont la réutilisabilité est parfois freinée par les dépendances fortes (comme l'héritage), les services offrent une meilleure interopérabilité et une plus grande flexibilité. Les composants distribués sont également apparus comme une tentative d'organisation logicielle plus robuste, mais c'est avec le Web Services Model que le paradigme s'est véritablement consolidé.

La littérature scientifique présente une diversité de modèles de services, incluant les services web, les services grid et les services destinés aux réseaux domestiques. Néanmoins, le modèle des services web¹ s'impose comme l'approche prédominante dans ce domaine. La conceptualisation des services web a donné lieu à diverses définitions d'une précision variable selon les auteurs. IBM [27] propose de caractériser un service web comme « une entité logicielle autonome et modulaire, accessible via un réseau pour satisfaire des exigences spécifiées ».

Austin et al. [2] proposent une approche plus technique en décrivant les services web comme « des applications identifiées des applications identifiées par un URI, dont l'interface et les liaisons sont découvrables, et qui communiquent avec d'autres applications par l'intermédiaire de messages XML ».

Booth David [7], propose une définition davantage axée sur les aspects techniques, mettant l'accent sur les interactions machine-à-machine réalisées via des

1. Dans la suite de notre mémoire nous utilisons les termes *service*, *service web* et *microservice* de manière interchangeable

messages SOAP transmis sur HTTP, avec une interface intelligible pour les systèmes automatisés.

L'analyse de ces différentes définitions révèle trois composantes essentielles qui définissent un service : (i) une interface explicite, décrivant les opérations disponibles et leurs signatures, (ii) un mécanisme de liaison (binding) définissant les messages échangés via un protocole de communication, et (iii) une aptitude à l'interaction avec d'autres services, autorisant la composition dynamique et distribuée.

À l'heure actuelle, les technologies orientées services ont connu une évolution considérable : Les Web Services classiques (SOAP/WSDL) ont laissé place à des approches plus légères, comme les services RESTful [16], utilisant HTTP et des formats JSON pour améliorer l'intégration avec le web moderne. Les outils de description d'API permettent de documenter et générer automatiquement des services. Les microservices, déployés via des conteneurs Docker et orchestrés avec Kubernetes, favorisent la scalabilité et la résilience applicative. Enfin, des architectures orientées événements (ex. AWS Lambda, Google Cloud Functions) [22] repoussent encore plus loin la granularité et la souplesse des services.

L'évolution vers une organisation par services autonomes, découplés et réactifs, est aussi facilitée par la montée des paradigmes orientés données, comme les Guarded Attribute Grammars (GAG) [4], qui permettent une exécution guidée par les dépendances de données et une composition incrémentale.

I.2. Calcul Orienté Services

I.2.1. Architecture Orientée Services

L'évolution du génie logiciel s'est caractérisée par l'émergence successive d'approches méthodologiques visant à répondre aux exigences croissantes du développement logiciel. Chaque nouveau paradigme proposé avait pour objectif de pallier aux limitations des paradigmes précédents en réutilisant certains principes et en créant de nouveaux. C'est ainsi que les approches orientées objets [50], orientées composants [46] et récemment les paradigmes orientés services ont vu le jour.

Les architectures basées sur les composants, telles que EJB et .NET ont été conçues pour masquer la complexité liée à l'intégration d'applications autonomes et hétérogènes [52]. Toutefois, ces technologies de composants distribués présentent un couplage relativement fort entre les objets, chaque architecture imposant sa propre infrastructure spécifique. Par conséquent, la mise en place de ces architectures dans un environnement ouvert comme Internet a présenté certaines difficultés.

Pour pallier ces contraintes, les efforts de recherche ont menés à l'émergence des architectures orientées services qui offrent une approche plus flexible et souple.

I.2.1.1. Définition

Il n'existe pas officiellement de définitions standards des SOA ; néanmoins nous avons relevé de la littérature quelques unes en référence à un ou plusieurs critères architecturaux :

Définition 1 *Une architecture orientée services constitue une architecture applicative dans laquelle l'ensemble des fonctionnalités sont conçues comme des services autonomes, dotés d'interfaces clairement définies et invocables pour l'élaboration de processus métier. Cette approche permet de développer des systèmes logiciels qui fournissent des services à d'autres applications par l'intermédiaire d'interfaces publiées et découvrables, ces services étant accessibles via un réseau [9].*

Définition 2 *La SOA est un modèle qui permet d'organiser et d'exploiter des ressources distribuées provenant de divers domaines. Elle offre un moyen standardisé pour proposer, découvrir, interagir et utiliser ces ressources afin d'atteindre le résultat souhaité avec des préconditions et des objectifs mesurables [23].*

Définition 3 *La SOA implique de diviser l'activité métier en une série de services qui peuvent interagir entre eux [38]. Ces services, ainsi que leurs descriptions et les opérations fondamentales (telles que la publication, la découverte et la liaison) qui produisent ou utilisent ces descriptions, forment la base de la SOA [37].*

À partir de ces énoncés, il ressort que la SOA représente une approche de conception logicielle qui utilise les services comme unité de base pour construire des systèmes distribués modulaires, flexibles et interopérables. Ces services, porteurs de fonctionnalités spécialisées, sont développés selon un principe d'indépendance technologique vis-à-vis des plateformes, langages et systèmes d'exploitation. Ils servent à répondre aux attentes des utilisateurs finaux ou à constituer des services composés plus complexes, encourageant la réutilisabilité, la scalabilité et l'intégration multi-environnements.

I.2.1.2. Propriétés

En général, dans une SOA, les services conformes sont censés répondre aux exigences suivantes [37, 27, 38, 15] :

- **Modularité** : chaque service est conçu de manière autonome pour accomplir une tâche spécifique et s'expose via une interface standardisée (REST, SOAP), facilitant ainsi son intégration dans des applications plus vastes ;
- **Couplage faible** : Dans une SOA, un client peut invoquer un service sans connaître ses détails internes, tels que la structure des données ou la base de données utilisée. De la même manière, le fournisseur de services n'a pas besoin d'accéder à l'état interne du client pour fournir une réponse. Les interactions entre eux reposent exclusivement sur des protocoles standards et ouverts, garantissant une communication indépendante et flexible ;
- **Réutilisabilité** : les services web existant peuvent être agrégés dans d'autres services composites. En effet, la modularité et le couplage faible des services favorise leur réutilisation ;
- **Interopérabilité** : Les services web sont capables de collaborer efficacement pour atteindre un objectif commun, quelque soit le contexte ou l'environnement dans lequel ils évoluent. Les services web sont développés pour intégrer les applications issues d'environnement distribués et hétérogènes ;
- **Composabilité** : Des services simples peuvent être associés afin d'effectuer des processus métier plus complexes ;
- **Communication Asynchrone** : les services interagissent sans que le consommateur de services ait besoin d'attendre une réponse immédiate pour continuer son exécution, permettant ainsi un traitement indépendant et parallèle.

1.2.1.3. Modèle SOA

Le modèle des SOA (Figure 1), est un modèle triangulaire constituée de trois entités principales : le **fournisseur**, le **client** et l'**annuaire** de services. Ces entités se chargent de la description, la publication, le stockage, la localisation et l'innovation de services indépendants disponibles à travers la toile.

Le fournisseur de service : un fournisseur conçoit un service web et en publie la description dans un répertoire de services. Cette description expose les fonctionnalités disponibles et leurs méthodes d'invocation, incluant les paramètres d'entrée nécessaires à chaque opération ainsi que les paramètres de sortie produits lors de l'exécution.

L'annuaire de service : un annuaire de services s'apparente à un répertoire centralisé référençant les services accessibles. Il assure la réception et l'enregistrement des spécifications de services publiées par les fournisseurs, ainsi que le traitement des demandes de recherche formulées par les clients.

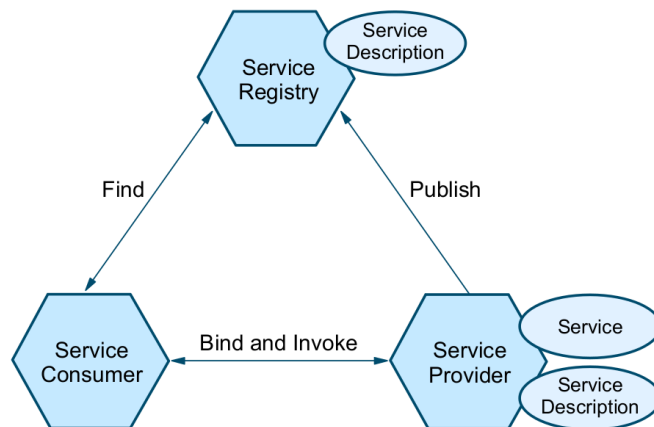


FIGURE 1 – Architecture Orientée Service [27, 38]

Le client : Le client peut identifier les services répondant à ses exigences par consultation du catalogue. Une fois que le catalogue a retourné la liste des descriptions de services disponibles, le client sélectionne le service le plus approprié et se connecte au fournisseur pour exécuter les opérations nécessaires. Il est important de noter que ce sont les opérations de service qui sont exécutées, et non les services eux-mêmes. Ainsi, parler d’invocation de service est en réalité un abus de langage.

1.2.2. Évolution des microservices

1.2.2.1. Définition

Un microservice est une unité logicielle autonome qui réalise une fonctionnalité métier spécifique, déployée indépendamment et capable de communiquer avec d’autres microservices par des interfaces légères. Cette architecture s’appuie sur le principe de *découplage maximal* : chaque service possède sa logique métier spécifique, sa base de données dédiée (le cas échéant), et peut faire l’objet de mises à jour, de déploiements ou de remplacements sans affecter le reste du système [36].

Définition 4 Selon Dragoni et al. [15], un microservice peut être défini comme un service à granularité fine, souvent construit autour d’un domaine métier précis (au sens de *Domain-Driven Design*), et pouvant s’intégrer dynamiquement dans une architecture distribuée.

Ces services interagissent entre eux par des mécanismes de communication légers et standardisés, notamment les API REST [45] ou encore gRPC [19].

1.2.2.2. Différence entre SOA et microservices

Alors que la SOA et les microservices partagent des principes fondamentaux similaires en matière de modularité et de découplage, ils se distinguent tout de même

par leur approche, leur portée et leurs modalités d'implémentation. Cette section présente une analyse comparative détaillée de ces deux approches architecturales, s'appuyant sur les études récentes qui mettent en évidence leurs différences structurelles et opérationnelles.

Comme le soulignent les recherches contemporaines, la principale distinction réside dans leur champ d'application : la SOA a une portée d'entreprise, tandis que l'architecture microservices a une portée d'application. Cette différence fondamentale influence tous les autres aspects de ces architectures.

Voici un tableau récapitulatif des principales différences [3, 18] :

TABLE I – Comparaison entre SOA et Microservices

	SOA	Microservices
Granularité	Services métiers à large portée	Services très spécialisés
Mise en œuvre	Différents services avec des ressources partagées.	Plus petits services indépendants et destinés à un objectif spécifique.
Communication	Un ESB utilise plusieurs protocoles de messagerie comme SOAP.	API, service de messagerie, Pub/Sub
Stockage de données	Stockage de données partagé.	Stockage de données indépendant.
Déploiement	Difficile. La reconstruction complète est nécessaire pour les petites modifications.	Facile à déployer. Chaque microservice peut être conteneurisé.
Réutilisabilité	Des services réutilisables s'appuyant sur des ressources communes partagées.	Chaque service dispose de ses propres ressources indépendantes. Vous pouvez réutiliser les microservices par l'intermédiaire de leurs API.
Rapidité	Ralentie à mesure que de nouveaux services sont ajoutés.	Vitesse constante à mesure que le trafic augmente.
Flexibilité de gouvernance	Gouvernance cohérente des données pour tous les services.	Des politiques de gouvernance des données différentes pour chaque stockage.

1.2.2.3. Avantages et défis des microservices

L'architecture microservices, bien qu'offrant de nombreux bénéfices en termes de flexibilité et d'évolutivité, présente également des défis considérables qui doivent être soigneusement évalués.

Avantages des Microservices

- **Modularité** : Chaque microservice correspond à une fonctionnalité précise, ce qui favorise une séparation claire des responsabilités et rend le système plus lisible et maintenable ;
- **Déploiement indépendant** : les microservices peuvent être implémentés, testés, déployés et mis à jour individuellement sans impacter le reste du système ;
- **Flexibilité technologique** : les microservices sont développés dans le langage ou la technologie la plus adaptée à leur besoin, tant qu'il respecte les protocoles d'échange.

Défis des microservices

- **Complexité opérationnelle** : le déploiement, le monitoring, la mise en réseau et l'administration de multiples services indépendants demandent une infrastructure complexe ;
- **Communication et réseau** : les appels inter-services sont souvent faits via le réseau, ce qui introduit de la latence, des risques de panne, et complexifie le contrôle des transactions distribuées ;
- **Consistance des données** : le modèle distribué rend le pilotage des données plus délicate.

Les microservices représentent une approche architecturale puissante qui peut apporter une valeur significative dans le bon contexte. Cependant, leur adoption nécessite une évaluation approfondie des capacités organisationnelles, techniques et financières. Le succès de cette architecture dépend largement de la maturité de l'organisation et de sa capacité à gérer la complexité distribuée.

I.3. Composition des Services

La composition des services est le processus qui permet d'intégrer des services dans une application. Le produit de cette composition est un nouveau service, appelé service composite. Dans ce contexte, la composition est qualifiée de récursive ou hiérarchique.

Actuellement, la composition est un domaine d'intérêt majeur pour la communauté scientifique ainsi que pour le secteur industriel. De nombreuses recherches se concentrent sur le développement de modèles de composition de services et sur la fourniture des outils nécessaires à cette fin.

Un service web est caractérisé comme *composé* ou *composite* dès lors que son fonctionnement implique des échanges avec d'autres services web en vue d'exploiter leurs capacités fonctionnelles. La composition de services web spécifie les

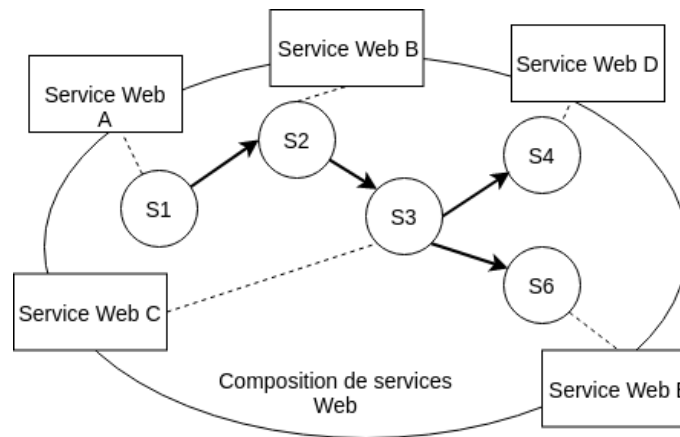


FIGURE 2 – Composition des services web [43]

services à invoquer, leur séquençement et les modalités de gestion des situations exceptionnelles.

I.3.1. Approches de composition de services

I.3.1.1. Composition statique

La composition statique se déroule en phase de conception applicative, impliquant la sélection, la liaison et l'assemblage préalables des composants. Cette méthode convient aux environnements stables à faible évolutivité. Elle génère cependant une rigidité applicative potentiellement incompatible avec les besoins clients. La composition statique des services web se décline en orchestration et chorégraphie [40].

Orchestration L'orchestration des services web requiert la définition d'un enchaînement selon un modèle établi et l'exécution via un script dédié. Les services web restent inconscients de la composition, seul l'orchestrateur maîtrisant cette connaissance. Le script et le modèle formalisent les interactions par l'identification des messages et la spécification des logiques d'invocation.

La figure 3 illustre une orchestration de service. Le service coordinateur contrôle tous les services web impliqué et coordonne l'exécution de leurs différentes opérations.

Chorégraphie À l'inverse de l'orchestration, la chorégraphie fonctionne sans coordinateur centralisé. Chaque service web participant possède une connaissance précise du moment d'exécution de ses opérations et des partenaires d'interaction concernés. Cette approche privilégie l'échange direct de messages entre services web plutôt que l'exécution d'un processus métier par une entité unique.

La chorégraphie constitue une démarche collaborative où chaque acteur du processus définit ses propres interactions. Elle établit l'enchaînement des échanges de

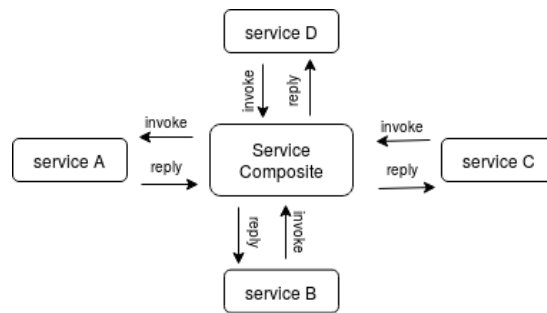


FIGURE 3 – Orchestration de services web [43]

messages impliquant potentiellement plusieurs services web. Cette collaboration chorégraphique peut être modélisée selon l'approche suivante (Figure 4) :

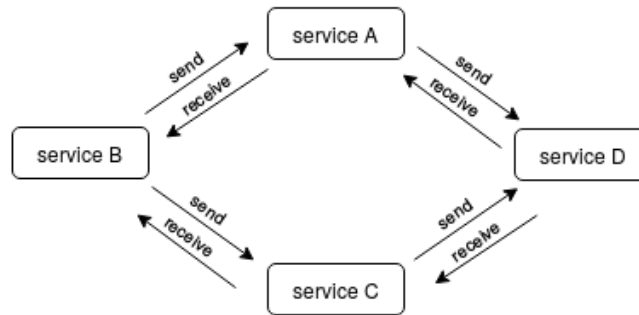


FIGURE 4 – Chorégraphie des services web [43]

I.3.1.2. Composition dynamique

La composition dynamique désigne l'assemblage de services web visant à satisfaire un objectif spécifique formulé par l'utilisateur selon ses préférences. Cette agrégation peut s'effectuer en amont ou durant l'exécution des services web. Actuellement, aucun standard n'encadre la spécification de cette composition dynamique de services web.

Les approches existantes de composition dynamique de services web peuvent être catégorisées selon : les *approches basées sur les workflows*, les *approches basées sur les techniques d'IA* et les *approches de spécification formelles*.

Nous constatons à travers le tableau II que les méthodes basées workflows sont inadaptées pour mener à bien le processus de composition des services web, alors que les approches basées IA et les approches issues des méthodes formelles offrent une spécification efficace de ce processus. Notons qu'un détail assez important échappe aux approches orientées IA, c'est la reconfiguration dynamique. Celle ci

TABLE II – Comparaison des approches de composition des services

Critères	Approches basées Workflows	Approches basées Intelligence Artificielle	Approches basées Méthodes Formelles
Niveau d'automatisation	-Moyen- Seule la recherche et la liaison aux services se font de manière automatique	-Élevé- Le processus est réalisé de manière automatique	Moyen-Élevé- Le Processus est partiellement automatisé
Niveau de dynamique	-Moyen- les modèles sont générés statiquement seule la liaison aux services est dynamique	-Élevé- possibilité de génération de plan de manière dynamique	-Élevé- Génération de modèles de composition graphiques de manière dynamique
Formulation de la requête utilisateur	-Bas niveau- (langage de flux)	Moyen à élevé Langages logiques	-Élevé- Spécification algébriques..
Support sémantique et niveau d'abstraction	-Bas niveau- (descriptions syntaxiques : WSDL, WSFL..)	-Élevé- Formalismes logiques, DAML-s, OWL-s.	-Élevé- Logique de réécriture, Algèbre de processus
Reconfiguration dynamique	non pris en charge	Non pris en charge	Envisageable

est prise en charge par la méthode formelle du fait de leur fondement sur les logiques computationnelles.

I.3.2. Vers la composition incrémentale

I.3.2.1. Motivations

Dans les architectures orientées services, la composition permet de construire des applications complexes à partir de services élémentaires. Jusqu'ici, deux grands paradigmes de composition sont largement utilisés : la composition statique, entièrement spécifiée lors de la phase de conception, et la composition dynamique, permettant la sélection ou l'adaptation de services durant l'exécution selon le contexte. Toutefois, ces deux approches supposent généralement que la structure globale du processus est à l'avance connue, et que l'ensemble des services nécessaires est disponible et sélectionné au moment du lancement du processus.

Dans les environnements modernes – *cloud computing*, *edge computing*, ou encore *systèmes collaboratifs distribués* – ces hypothèses ne tiennent plus toujours. Les données peuvent arriver de manière partielle et imprévisible, l'accessibilité immédiate de certains services n'est pas garantie, et la composition doit pouvoir évo-

luer de manière réactive face à ces contraintes. C'est dans ce contexte qu'émerge la composition incrémentale des services.

La composition incrémentale désigne une approche dans laquelle la construction et l'exécution du processus s'effectuent de manière progressive, selon la disponibilité des éléments (services, données, ressources) effectivement disponibles. Elle s'inscrit dans une logique data-driven, où l'activation des étapes d'un processus dépend directement des informations disponibles localement, et non d'un ordonnancement global préétabli. Les services sont ajoutés, remplacés ou connectés à la volée, tout au long de l'exécution, en maintenant l'état partiel du processus. La composition incrémentale conjugue granularité, suivi dynamique des dépendances, activation paresseuse des services et pilotage guidé par les données. Ces principes permettent de construire des processus flexibles, efficaces et adaptés aux contraintes des architectures modernes.

1.3.2.2. Principes

La composition incrémentale repose sur un changement fondamental par rapport aux approches statique et dynamique traditionnelles : elle considère que la construction du processus ne peut pas être entièrement planifiée à l'avance, mais doit s'adapter en continu aux informations et ressources qui deviennent disponibles au fil de l'exécution. Cette adaptation progressive nécessite plusieurs principes fondamentaux.

Premièrement, la décomposition fine du processus en sous-composants élémentaires permet une gestion granulaire de la composition. Chaque sous-service peut être vu comme une unité autonome, susceptible d'être activée indépendamment en fonction des données disponibles ou des événements reçus. Cette granularité facilite l'activation sélective des parties du workflow, évitant ainsi de lancer ou recomposer l'ensemble du processus pour chaque changement.

Ensuite, la composition incrémentale repose sur un mécanisme de suivi des dépendances entre services et données. Chaque étape du processus est liée à un ensemble précis d'informations d'entrée ; lorsqu'une donnée évolue ou qu'un service devient disponible, seule la partie concernée du processus est réévaluée ou étendue. Cette propagation ciblée des changements optimise les coûts de calcul et limite les traitements redondants.

Par ailleurs, la notion de services « paresseux » ou *lazy evaluation* joue un rôle cruciale. Plutôt que d'exécuter immédiatement chaque service dans le flux, la composition incrémentale peut différer l'activation de certains services jusqu'au moment où leurs résultats sont effectivement nécessaires. Cette stratégie retarde la

consommation des ressources et réduit les calculs inutiles, en s'adaptant à l'évolution des données en temps réel.

Enfin, la composition incrémentale s'inscrit dans un paradigme *data-driven* : l'ordre et la sélection des services dépendent directement des événements et des données locales, et non d'un plan global statique. Cette orientation conduit à des processus plus réactifs et modulaires, capables de s'adapter à des environnements dynamiques et incertains, comme ceux rencontrés dans le cloud, les systèmes distribués ou les flux continus.

1.3.2.3. Grammaires Attribuées Gardées (GAG)

La GAG [4] est un cadre formel inspiré des concepts de la grammaire attribuée de Knuth [26], également considérée comme grammaires de processus, et conçue pour modéliser des architectures ouvertes centrées sur l'utilisateur où les données et les règles nécessitent de la flexibilité.

Commençons par rappeler la notion d'une grammaire hors-contexte et d'une grammaire attribuée, qui sont à la base de la notion de Grammaires Attribuées Gardées

Définition 5 On appelle **grammaire hors-contexte** (CFG) le quadruplet (N, T, S, P) avec :

- N un ensemble fini de non-terminaux ;
- T un ensemble fini de terminaux, $N \cap T = \emptyset$;
- P un ensemble fini de production, $P \subset N \times (N \cup T)^*$ de la forme $A \rightarrow A_1 \dots A_n$ où $(A \in N \text{ et } A_i \in N \cup T)$;
- $S \in N$ est l'axiome de départ.

Définition 6 Une **grammaire attribuée** est un triplet (G, A, F) où :

- $G = (N, T, S, P)$ est une grammaire hors-contexte ;
- A est l'ensemble d'attributs défini par :

$$A = \bigcup_{X \in N} (H(X) \cup S(X)) \quad (\text{I.1})$$

avec $H(X)$ les attributs **hérités** de X ,

$S(X)$ les attributs **synthétisés** de X

et $H(X) \cap S(X) = \emptyset$

- F est l'ensemble des règles sémantiques associées aux productions.

Les GAG sont apparues à l'origine comme un instrument pour la compilation des langages fonctionnels, permettant l'analyse descendante avec des grammaires

contenant des productions gauches. Les stratégies d'analyse descendante permettent de décrire les programmes de manière très modulaire, tandis que les productions récursives à gauche permettent l'évaluation paresseuse de ces programmes. Cette modularité est une caractéristique intéressante pour une approche orientée service, et l'évaluation paresseuse est ce qui permet un calcul incrémental. En tant que modèle formel et plutôt abstrait, leur domaine d'application s'est élargi depuis lors.

Grammaires et workflow Les GAG sont fondamentalement des grammaires formelles. Elles intègrent structurellement un *alphabet*, divisé en symboles *terminaux* (T) et *non terminaux* (N), ainsi qu'un ensemble de *règles de production* ou simplement de *productions* ($r \in P$). Ces règles décrivent la réécriture des symboles non terminaux sous forme de chaînes de caractères sur l'alphabet. En réécrivant récursivement les symboles non terminaux, l'application d'un nombre fini de règles finit par produire une chaîne composée uniquement de symboles terminaux. Comme dans une grammaire hors-contexte, chaque règle décrit la réécriture d'un seul symbole non terminal isolé.

$$r \in P \quad \text{est} \quad \sigma \rightarrow \sigma_1 \dots \sigma_n \quad \text{avec} \quad \sigma \in N \quad \text{et} \quad \sigma_i \in N \cup T \quad (\text{I.2})$$

Cependant, la structure plus riche du modèle permet de décrire une certaine forme de contexte et la grammaire peut donc accepter des langages plus complexes que les simples langages sans contexte. En effet, chaque symbole de l'alphabet représente une fonction. À l'origine, les symboles étaient en fait destinés à représenter des transducteurs d'arbres, transformant l'arbre d'analyse d'une expression sous une forme exécutable. En général, les symboles peuvent être interprétés comme des tâches à accomplir ou des problèmes à résoudre. Ainsi, les règles décrivent comment une tâche (problème) peut être divisée en sous-tâches (sous-problèmes).

Une tâche peut impliquer le traitement d'informations, et ces informations peuvent influencer la manière dont la tâche est exécutée. Dans notre cadre, les symboles représentent des services, qui peuvent nécessiter des données d'entrée et produire des données de sortie, ainsi que d'éventuels effets secondaires, tels que l'interrogation d'une base de données ou l'exécution d'un morceau de code sur une micro-architecture particulière. Les symboles non terminaux représentent des services composites, et les symboles terminaux représentent des calculs locaux ou des services de tiers. Les règles de production spécifient ensuite le flux de travail de l'application. La grammaire doit être spécifiée à la compilation, mais la réécriture se fait à l'exécution. En effet, les données d'entrée détermineront quelle production doit réécrire un symbole donné.

Attributs et data-flow. Un service est désigné par (I, σ) où $I \in A$ représente les données d'entrée et σ le service invoqué. La notation (I, σ, O) indique qu'un appel de service σ avec les données d'entrée I a produit les données de sortie $O \in A$.

Les données d'entrée et de sortie doivent être structurées (comparable à des fichiers XML ou JSON) et sont formalisées par des attributs ($\alpha \in A$). Un attribut est conceptuellement une liste associant des champs arbitraires ($D \in F$) à des valeurs ($X \in D$). La valeur d'un champ peut elle-même être un attribut, créant ainsi des structures arborescentes.

Une caractéristique distinctive des attributs est la possibilité de laisser des valeurs indéfinies, appelées valeurs intentionnelles ($\perp$). Une valeur intentionnelle représente un engagement que la valeur réelle sera éventuellement définie, permettant ainsi un calcul incrémental.

Formellement :

$$\alpha \in A \quad \text{est} \quad [(D_1, x_1), \dots, (D_k, x_k)] \quad \text{avec} \quad D_i \in F \quad \text{et} \quad x_i \in D_i \cup \{A, \perp\} \quad (\text{I.3})$$

La structuration des données sous forme d'attributs est fondamentale dans les GAG car elle permet la correspondance de motifs nécessaire à la liaison des variables lors des réécritures. Les GAG constituent des systèmes de réécriture conscients des données où les règles de production dépassent la simple décomposition de services pour décrire explicitement les flux de données entre sous-services. Chaque production est associée à trois types de variables : les variables d'entrée ($x \in X$), les variables de sortie ($y \in Y$) représentant les valeurs retournées par le service, et les variables locales ($l \in L$). Ces variables servent d'espaces réservés lors de la compilation et sont remplacées par des valeurs (éventuellement intentionnelles) à l'exécution. Les flux de données sont formalisés par les règles sémantiques : chaque production $\sigma \rightarrow \sigma_1 \dots \sigma_n \in R$ est associée à une liste de règles sémantiques contenant une affectation pour chaque σ_i de la forme :

$$v \leftarrow \sigma_i w \quad \text{où} \quad v \in Y \cup L \quad \text{et} \quad w \in X \cup Y \cup L \quad (\text{I.4})$$

Les variables d'entrée sont en lecture seule (côté droit uniquement) tandis que chaque variable de sortie a exactement une affectation (côté gauche unique), les variables locales sont minimales pour optimiser les flux de données. Au moment de l'exécution, chaque règle sémantique s'exécute sur son propre thread et l'ordre d'exécution est déterminé par le flux de données disponibles. Lorsqu'une production est appliquée, chaque règle sémantique ($v \leftarrow \sigma_i w$) crée une disposition (I, σ_i, O) contenant des valeurs intentionnelles qui sont progressivement remplacées par des valeurs réelles au fur et à mesure que les demandes (I, σ_i) sont réso-

lues récursivement. Cette approche permet une définition incrémentielle des données structurées, où la résolution partielle d'une valeur peut déclencher la définition d'autres valeurs dans des dispositions associées, créant ainsi des flux de données complexes. Le mécanisme d'association des valeurs repose sur la correspondance de motifs effectuée par les gardes : lors d'une demande de service (I, σ) , chaque production applicable associe les valeurs d'entrée à ses variables via un pattern matching, où chaque variable d'entrée correspond à un champ spécifique de l'attribut I .

Gardes et flux de contrôle. Les gardes constituent le mécanisme de contrôle des Grammaires Attribuées Gardées. Chaque production $r \in R$ possède une garde $g \in G$ qui évalue si l'attribut d'entrée I contient les champs nécessaires à l'activation de la production. Cette évaluation peut être récursive lorsque les champs sont eux-mêmes des attributs.

Une production r n'est applicable que si sa garde génère avec succès une affectation $x_i \leftarrow v_i$ pour chaque variable d'entrée x_i , où v_i provient de I . Les gardes étant évaluées dynamiquement, des productions peuvent devenir applicables au fur et à mesure que les valeurs intentionnelles se définissent. Pour maintenir le déterminisme, l'application d'une production pour résoudre (I, σ) désactive automatiquement toutes les autres productions candidates.

Les gardes permettent aux services d'avoir des décompositions alternatives selon les données d'entrée, encodant ainsi le flux de contrôle de l'application et autorisant des définitions récursives. Elles garantissent un traitement sécurisé et récursif des données en appliquant le même service sur des portions progressivement réduites de l'entrée, assurant ainsi la terminaison du processus.

I.4. Distribution des services

Dans les architectures logicielles modernes, et plus particulièrement dans les environnements cloud-native, l'exécution des services ne se limite plus à leur simple implémentation fonctionnelle. Elle dépend étroitement de l'infrastructure technique qui permet de les déployer, de les superviser, et de les faire évoluer dynamiquement. Cette infrastructure repose aujourd'hui sur deux piliers fondamentaux : la conteneurisation et l'orchestration.

La conteneurisation, rendue populaire par Docker [34], permet d'empaqueter les services avec leurs dépendances dans des unités légères, isolées et portables appelées conteneurs. Cette approche a profondément transformé la manière de dé-

velopper, de livrer et d'exécuter les applications, en supprimant les problèmes de compatibilité liés aux environnements d'exécution.

Cependant, à mesure que les systèmes distribués deviennent plus complexes et que le nombre de conteneurs augmente, il devient nécessaire de disposer d'un mécanisme capable de les gérer de manière automatisée. C'est le rôle des plateformes d'orchestration de conteneurs, dont Kubernetes [6] s'est imposé comme le standard de facto. Ces plateformes assurent le déploiement, la montée en charge, la résilience et la supervision des services conteneurisés, en facilitant leur exécution distribuée à grande échelle.

Dans le cadre de la composition incrémentale des services, ces technologies sont particulièrement pertinentes. Elles permettent d'exécuter les services de manière indépendante, réactive et scalable, en réponse à des données ou événements, tout en assurant la robustesse et la flexibilité nécessaires à une architecture orientée événements et pilotée par les données.

Cette section présente dans un premier temps les fondements de la conteneurisation, avant d'introduire les principales caractéristiques des orchestrateurs de conteneurs, avec un focus particulier sur Kubernetes, son fonctionnement, et ses avantages comparés aux autres solutions existantes.

I.4.1. Le cloud computing

Le cloud computing est une infrastructure informatique distribuée qui permet d'accéder à des ressources de calcul et de stockage via un réseau, généralement Internet. Selon le NIST, il s'agit de « l'accès via le réseau, à la demande et en libre-service, à des ressources informatiques virtualisées et partagées » [33].

Cette approche permet aux utilisateurs d'accéder à leurs applications et données depuis n'importe quel appareil connecté, sans se soucier de l'infrastructure sous-jacente.

Caractéristiques du *cloud*. Le cloud computing se distingue des infrastructures informatiques classiques par certaines caractéristiques :

- **Services à la demande** : Les clients peuvent se fournir automatiquement des services sans nécessiter d'intervention de la part du fournisseur ;
- **Élasticité rapide** Les ressources peuvent être ajustées de manière statique ou dynamique selon les variations de charge. Cette capacité de montée en charge (scaling) permet d'adapter instantanément l'infrastructure aux pics de demande tout en optimisant les coûts lors des périodes creuses ;
- **Mutualisation** : partage optimisé des ressources de serveur allouées entre plusieurs serveurs ;

- **Résilience** : Compatibilité avec tous types d'appareils, des smartphones aux ordinateurs de bureau ;
- **Païement à l'utilisation** : Facturation transparente basée sur la consommation réelle des ressources, remplaçant les forfaits prépayés traditionnels.

Les niveaux de services cloud. Le NIST définit trois modèles de service cloud qui correspondent à différents niveaux d'abstraction et de responsabilité entre le fournisseur et l'utilisateur : le IaaS, PaaS, SaaS [33, 22, 41].

- **Infrastructure as a Service** : Le fournisseur met à disposition l'infrastructure informatique de base (serveurs virtuels, stockage, réseaux). L'utilisateur conserve le contrôle du système d'exploitation, des applications et des configurations réseau. *Exemples : Amazon EC2, Microsoft Azure VMs, Google Compute Engine ;*
- **Platform as a Service** : Le fournisseur offre une plateforme de développement complète incluant l'infrastructure, le système d'exploitation, les middlewares et les outils de développement. L'utilisateur se concentre uniquement sur le développement et le déploiement de ses applications. *Exemples : Google App Engine, Heroku, Microsoft Azure App Service ;*
- **Software as a Service** : Le fournisseur livre des applications complètes et opérationnelles accessibles via un navigateur web ou une API. L'utilisateur n'a aucune responsabilité sur l'infrastructure ou la maintenance. *Exemples : Gmail, Google Docs, Microsoft 365, Dropbox.*

Ces trois modèles traditionnels ont évolué avec l'émergence du Function as a Service (FaaS), qui représente le niveau d'abstraction le plus poussé. Le FaaS permet l'exécution de fonctions individuelles sans aucune gestion de serveurs, où les développeurs déploient uniquement du code métier tandis que la plateforme gère automatiquement l'allocation des ressources et la facturation au temps d'exécution (*AWS Lambda, Azure Functions, Google Cloud Functions*). Cette évolution vers des granularités de plus en plus fines a directement inspiré le développement des technologies de conteneurisation, qui offrent une approche intermédiaire entre la flexibilité de l'IaaS et la simplicité du PaaS.

La figure 5 illustre la répartition des responsabilités entre fournisseur et client selon les niveaux de service cloud. Ces modèles ne constituent du véritable cloud computing que s'ils respectent les caractéristiques essentielles présentées précédemment, sans lesquelles il s'agirait simplement d'hébergement traditionnel.

Modèles de déploiement. Les niveaux de services cités peuvent être déployés suivant quatre modèles :

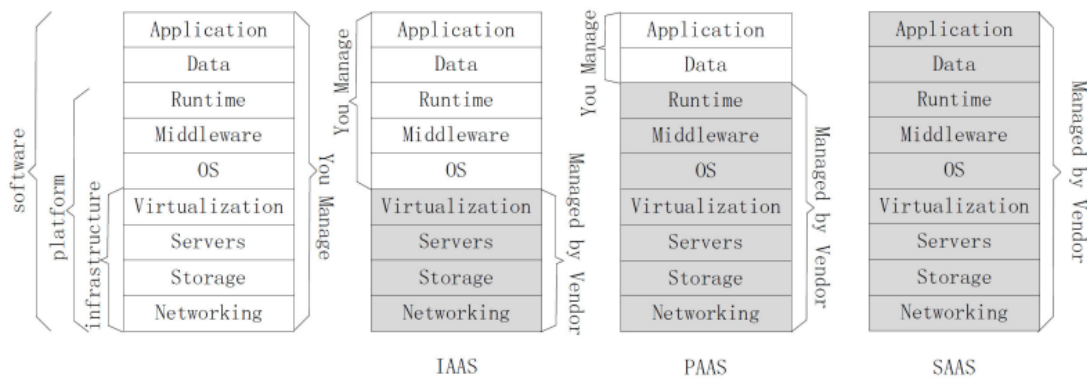


FIGURE 5 – Niveaux de services dans le cloud [41]

- **Cloud public** : Infrastructure gérée par un fournisseur tiers (*Amazon Web Services, Google Cloud Platform, Microsoft Azure*) et partagée entre plusieurs organisations. Les ressources sont accessibles via Internet avec une facturation à l'usage ;
- **Cloud privé** : Infrastructure dédiée exclusivement à une organisation, pouvant être (i) **interne** : l'entreprise gère son propre centre de données, (ii) **externe** : un fournisseur réserve des ressources dédiées accessibles via des connexions sécurisées (VPN) ;
- **Cloud communautaire** : Infrastructure partagée entre plusieurs organisations ayant des préoccupations communes (*sécurité, conformité, juridiction*). Peut être géré par l'une des organisations participantes ou par un tiers ;
- **Cloud hybride** : C'est une combinaison de deux ou plusieurs modèles de déploiement (privé, communautaire, public) qui restent des entités distinctes mais sont interconnectées par des technologies standardisées ou propriétaires. Cette approche facilite la portabilité des données et des applications entre différents environnements selon les besoins de l'entreprise.

Le défi de l'isolation dans le cloud. La mutualisation des ressources cloud nécessite des mécanismes d'isolation pour garantir :

- **Exclusivité d'accès** : ressources dédiées malgré l'infrastructure partagée ;
- **Transparence** : illusion d'être le seul utilisateur du système ;
- **Équité** : prévention de la monopolisation par un client unique ;
- **Étanchéité** : protection des données entre espaces utilisateurs

I.4.2. La virtualisation

Définition La virtualisation consiste à créer une représentation logicielle d'une ressource physique, qu'il s'agisse d'un système d'exploitation, d'un serveur, d'un dispositif de stockage ou d'une infrastructure réseau [13, 31].

Cette technologie opère par abstraction des ressources physiques : les composants virtuels (processeurs, mémoire, disques, interfaces réseau) correspondent à des ressources matérielles réelles du système hôte, mais sont présentés de manière abstraite aux environnements virtualisés. Ainsi, chaque machine virtuelle dispose de ses propres ressources virtuelles, mappées dynamiquement sur les ressources physiques disponibles.

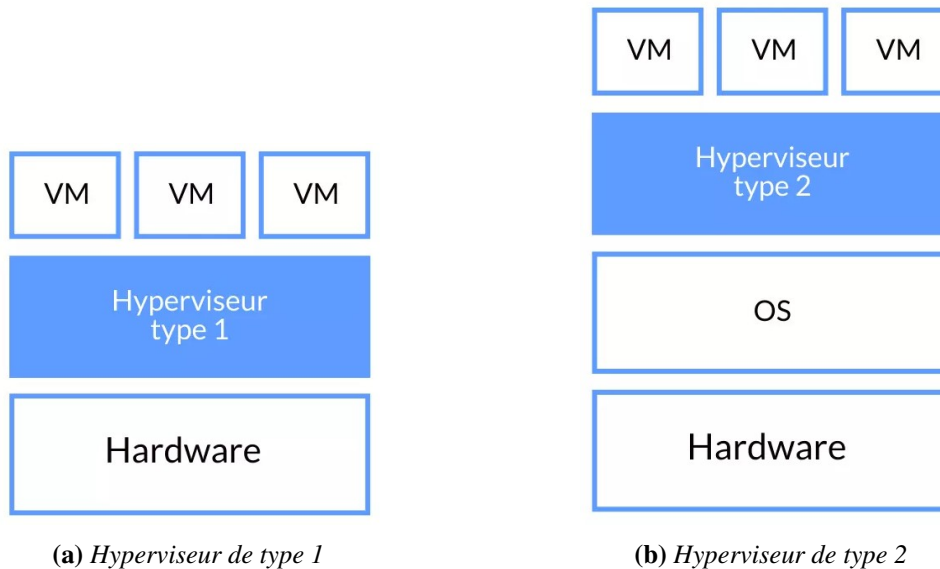
Du point de vue du système hôte, les machines virtuelles s'exécutent comme des applications ordinaires auxquelles des ressources sont allouées et distribuées selon les besoins. Cette approche permet de faire fonctionner simultanément plusieurs environnements isolés sur une même infrastructure physique.

Virtualisation système. La virtualisation système repose sur un hyperviseur, également connu sous le nom de VMM (Virtual Machine Monitor), pour permettre à plusieurs machines virtuelles (VM) de fonctionner simultanément sur une seule infrastructure physique. Ces concepts sont donc définis de la manière suivante :

- **Machine Virtuelle (VM)** : Environnement virtualisé reproduisant un système informatique complet avec son propre système d'exploitation. Chaque VM gère de manière autonome l'accès à ses ressources virtuelles (*processeur, mémoire RAM, stockage, réseau*) tout en fonctionnant comme une machine physique indépendante ;
- **Hyperviseur (VMM)** : Couche logicielle qui implémente les mécanismes d'isolation et de partage des ressources matérielles. Il orchestre l'exécution simultanée de plusieurs VM de types différents (*Linux, Windows, macOS*) sur le même matériel physique. L'hyperviseur contrôle l'allocation des ressources, gère la planification CPU et configure l'accès réseau des VM via l'attribution d'adresses IP et la mise en place de politiques de routage et de filtrage. On peut distinguer deux types d'hyperviseur (i) **Hyperviseur de type 1 (natif)** : il s'installe directement sur le matériel physique et prend le contrôle exclusif des ressources au démarrage. Cette approche offre des performances optimales et une latence réduite. *Exemples : VMware ESXi, Microsoft Hyper-V, Citrix Xen, KVM* ; (ii) Celui ci est similaire à un émulateur, il s'exécute comme une application sur un système d'exploitation hôte existant. Plus simple à déployer mais avec des performances moindres

dues à la couche OS supplémentaire. *Exemples : Oracle VirtualBox, VMware Workstation, Parallels Desktop.*

Les figures 6a et 6b donne un aperçu sur les deux types d'hyperviseur de la virtualisation système.



La virtualisation applicative. La virtualisation applicative adopte une approche différente de la virtualisation système en implémentant une couche de virtualisation au niveau applicatif plutôt qu'au niveau matériel. Cette technique crée des instances virtuelles isolées des spécificités de la machine hôte (architecture, système d'exploitation), permettant aux développeurs de créer des applications portables sans devoir maintenir plusieurs versions pour différents environnements. Les VM applicatives les plus connues sont la *JVM* et les *conteneurs*.

- **Java Virtual Machine (JVM) :** environnement d'exécution qui permet aux applications Java compilées de produire des résultats identiques sur toutes les plateformes disposant de la JVM appropriée. Cette approche "write once, run anywhere" a révolutionné le développement d'applications multiplateformes. La JVM était propriétaire jusqu'à la version 6, puis est devenue libre sous licence GPL à partir de la version 7 [21];
- **Conteneurs :** technologie de virtualisation applicative offrant une portée plus large que la JVM. Les conteneurs supportent multiple langages et frameworks (*Java, Python, Ruby, Node.js, Go*) tout en étant compatibles avec l'écosystème Linux. Ils fournissent un niveau d'isolation entre instances tout en optimisant l'utilisation des ressources système (CPU, RAM, stockage) de la machine hôte.

I.4.3. Virtualisation par conteneurs

Définition Également appelée *conteneurisation*, la virtualisation par conteneurs constitue une alternative légère à la virtualisation système traditionnelle. C'est une approche qui exploite directement les fonctionnalités natives du noyau Linux (*Kernel*) pour créer des environnements virtuels (VE - Virtual Environment) appelés *conteneurs*, isolés les uns des autres.

La virtualisation par conteneur repose sur deux fonctionnalités clés du noyau Linux, les groupes de contrôles (*cgroups*) et les espaces de noms (*namespaces*) [35]. Les *namespaces* permettent l'isolation qui partitionnent les ressources système (processus, réseau, système de fichiers, utilisateurs) pour créer des vues distinctes et isolées pour chaque conteneur. Chaque conteneur dispose ainsi de sa propre vision du système sans interférer avec les autres. Tandis que les *cgroups* sont des fonctionnalités de limitation et de surveillance qui contrôlent et régulent la quantité de ressources (CPU, mémoire, I/O) allouée à un groupe de processus. Les *cgroups* garantissent un partage équitable des ressources et empêchent qu'un conteneur monopolise les capacités du système hôte.

Vue d'ensemble. Les conteneurs informatiques s'inspirent des conteneurs d'expédition pour standardiser le déploiement d'applications. Leur évolution historique suit plusieurs étapes clés :

- **1979 - Unix chroot** : Premier mécanisme d'isolation créant des sessions isolées avec partage des ressources, précurseur de la conteneurisation ;
- **2000 - FreeBSD Jail** : Partitionnement du système d'exploitation en sous-systèmes isolés avec isolation étendue (système de fichiers, utilisateurs, réseau) ;
- **2001 - Linux VServer** : Solution VPS (Virtual Private Server) partitionnant les ressources en contextes de sécurité ;
- **2008 - Linux Containers (LXC)** : Exploitation optimale des fonctionnalités du noyau Linux, mais avec des problèmes de compatibilité entre distributions ;
- **2013 - Docker** : Standardisation et simplification de la conteneurisation, résolvant le problème de portabilité des applications dans le cloud computing.

Les conteneurs constituent une solution efficace pour la gestion des ressources entre applications [35]. L'importance de la portabilité dans le contexte cloud justifie notre focus sur la technologie *Docker* dans la suite.

Les conteneurs Docker Docker représente l'évolution moderne de la technologie de conteneurisation. Initialement développé par la société *Dotcloud* (devenue

Docker Inc.), ce projet open source s'appuyait originellement sur la technologie *LXC*.

L'innovation principale de Docker réside dans l'introduction du *Docker daemon*, une API qui s'intercale entre le noyau du système d'exploitation et les environnements virtuels [24]. Cette couche intermédiaire permet l'exécution de processus complètement isolés, gérant de manière indépendante les ressources système : processeur, mémoire, entrées/sorties et réseau. À partir de la version 0.9 sortie en 2014, Docker a abandonné sa dépendance à *LXC* pour développer sa propre solution : *Libcontainer* [8]. Cette bibliothèque propriétaire, programmée en langage *Go*, offre un environnement d'exécution optimisé spécifiquement pour l'écosystème Docker.

Docker se distingue par son approche complète et intuitive du *développement conteneurisé*. L'écosystème propose un ensemble d'outils intégrés comprenant un système de gestion d'images sophistiqué, des registres pour le stockage et la distribution, ainsi qu'une interface en ligne de commande conviviale. Cette architecture cohérente fait de Docker une plateforme complète pour la conteneurisation moderne.

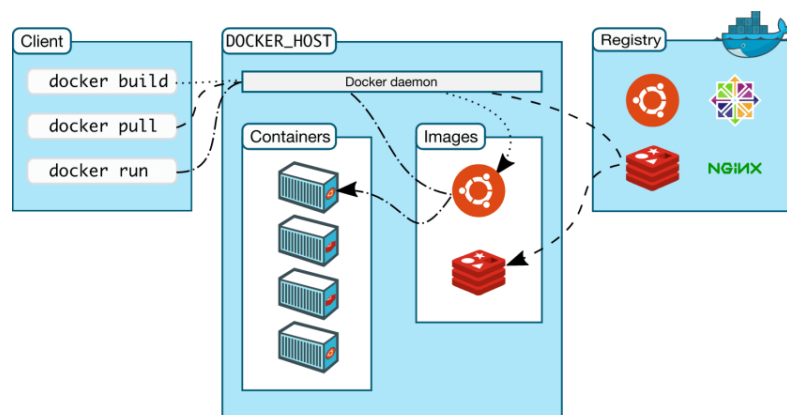


FIGURE 7 – Architecture de Docker [14]

La figure 7 présente l'architecture de docker,

- **Le client Docker** : Interface en ligne de commande permettant aux utilisateurs d'émettre des commandes (push, pull, run, build, etc.) vers le daemon Docker. Il constitue le point d'entrée principal pour interagir avec l'écosystème Docker ;
- **Le daemon Docker (dockerd)** : Composant central qui gère l'ensemble des conteneurs sur le système hôte. Il traite les requêtes du client Docker, supervise le cycle de vie complet des conteneurs (création, démarrage, arrêt,

- suppression) et assure la communication avec d'autres daemons pour les services distribués ;
- **Les images Docker** : Modèles en lecture seule contenant l'application et toutes ses dépendances (bibliothèques, fichiers de configuration, variables d'environnement) ;
 - **Les conteneurs Docker** : Instances d'images en cours d'exécution. Chaque conteneur constitue un environnement isolé du système hôte et des autres conteneurs, disposant de son propre espace processus, réseau et système de fichiers ;
 - **Le Registry** : Service de stockage et de distribution des images Docker, pouvant être public (comme Docker Hub) ou privé (registry d'entreprise). Il permet le partage, la versioning et la distribution centralisée des images entre différents environnements.

I.4.4. Comparaison entre conteneur Docker et VMs

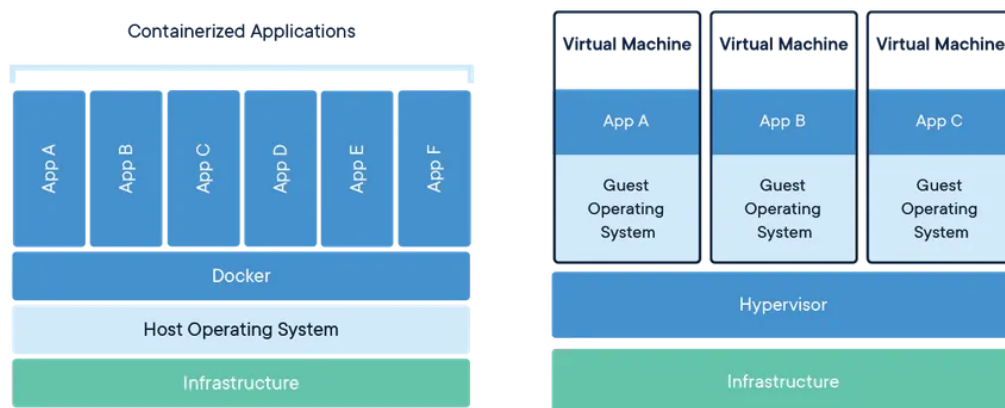


FIGURE 8 – Comparaison de l'architecture d'une machine virtuelle et d'un conteneur Docker [14]

La virtualisation traditionnelle s'appuie sur un hyperviseur pour créer et isoler les machines virtuelles, tandis que la conteneurisation Docker ne requiert que l'installation du moteur Docker sur le système d'exploitation hôte. Malgré leurs différences techniques, ces deux technologies partagent un point commun : elles créent des environnements virtuels autonomes qui exploitent les ressources d'un système hôte supérieur. La distinction fondamentale réside dans leur approche de l'isolation : les machines virtuelles embarquent un système d'exploitation complet (OS invité), alors que les conteneurs partagent le noyau du système d'exploitation hôte 8.

Les conteneurs Docker, évolution des capacités LXC, n'émulent pas l'environnement matériel sous-jacent (Felter et al., 2015 ; Naik, 2016). Cette approche per-

met un déploiement flexible aussi bien sur machines physiques que virtuelles. Les conteneurs offrent des performances supérieures aux machines virtuelles : création et destruction quasi-instantanées avec une surcharge négligeable des ressources système (CPU, RAM, E/S) [21]. Ils excellent également en termes de gestion réseau, consommation mémoire, stockage, vitesse de démarrage, déploiement et migration.

Cependant, les conteneurs présentent un niveau d'isolation inférieur aux machines virtuelles. Une compromission d'un conteneur Docker peut potentiellement donner à un attaquant un accès privilégié au système d'exploitation hôte. Cette vulnérabilité souligne l'importance de déployer les conteneurs dans des environnements robustes et sécurisés. Le tableau III présente une étude comparative entre la virtualisation système (VM) et la virtualisation par conteneur.

TABLE III – *Comparaison entre machines virtuelles et conteneurs Docker*

Critères	Machine Virtuelle	Conteneur Docker
OS	Chaque VM virtualise complètement le matériel et intègre son propre système d'exploitation	Les conteneurs n'émulent pas le matériel physique et utilisent directement l'OS de la machine hôte
Communication	Communication via les périphériques Ethernet virtualisés	Utilise les mécanismes IPC standards : sockets, pipes, mémoire partagée, etc.
Consommation des ressources (CPU et RAM)	Forte consommation des ressources système	Consommation quasi-native avec une surcharge minimale
Temps de démarrage	Démarrage en plusieurs minutes	Démarrage en quelques secondes seulement
Stockage	Nécessite un espace de stockage important car l'OS complet et les applications associées doivent être installés et exécutés	Utilise moins d'espace de stockage grâce au système de couches partagées
Portabilité	Le partage de bibliothèques et fichiers entre VMs est complexe	Les sous-répertoires peuvent être montés de manière transparente et facilement partagés
Sécurité	Isolation robuste dépendant de la configuration de l'hyperviseur	Nécessite un contrôle d'accès renforcé en raison du partage du noyau

Cette section a exploré les fondements de la conteneurisation dans le contexte du cloud computing. L'analyse comparative démontre la supériorité des conteneurs Docker sur les machines virtuelles en termes de performances, consommation de

ressources et vitesse de déploiement. Bien que Docker excelle dans la gestion de conteneurs sur une machine unique, il présente des limitations pour les déploiements distribués multi-machines. Cette lacune a conduit au développement de plateformes d'orchestration, sujet de la section suivante, permettant de gérer efficacement des architectures conteneurisées complexes à grande échelle.

1.5. Orchestration des conteneurs

La conteneurisation Docker a révolutionné le développement et le déploiement d'applications grâce à sa légèreté et sa flexibilité. Cette technologie a favorisé l'émergence d'architectures distribuées où les applications sont organisées en conteneurs déployables sur des clusters de machines. Cette évolution nécessite de nouveaux outils spécialisés : les orchestrateurs de conteneurs, essentiels pour gérer la complexité des déploiements distribués à grande échelle [6].

La gestion de quelques conteneurs Docker sur une machine unique reste simple. Cependant, le passage en production sur des infrastructures distribuées soulève de nombreux défis critiques : *gestion des déploiements, équilibrage de charge, communication inter-conteneurs, découverte de services, mises à jour, montée en charge, persistance des données, configuration des secrets et gestion des pannes*.

Une approche manuelle de ces problématiques compromettrait la viabilité et la pérennité du système. D'où la nécessité d'une plateforme d'orchestration automatisée. Plusieurs solutions existent [44] : *Fleet, Apache Mesos, Docker Swarm* et *Kubernetes*. Cette section se concentre sur Kubernetes, solution stable et gratuite qui automatise le déploiement, la migration, la surveillance, la mise en réseau, l'extensibilité et la disponibilité des applications conteneurisées.

Définition Kubernetes (K8s) est un orchestrateur de conteneurs initié par Google en 2014, fruit de plus d'une décennie d'expérience avec Borg, leur gestionnaire de conteneurs interne [6]. Cette plateforme automatise le déploiement et la gestion d'applications conteneurisées à grande échelle sur des clusters de machines physiques et/ou virtuelles. Kubernetes gère l'intégralité du cycle de vie des applications en garantissant prévisibilité, extensibilité et haute disponibilité.

Architecture de Kubernetes

Comme le montre la figure 9, l'architecture Kubernetes suit un modèle maître-esclave composé de deux types de nœuds principaux [28]. Le nœud **maître** centralise la gestion et le contrôle du cluster à travers quatre composants essentiels :

- **API Server** : point d'entrée central exposant l'API REST de Kubernetes pour toutes les interactions ;

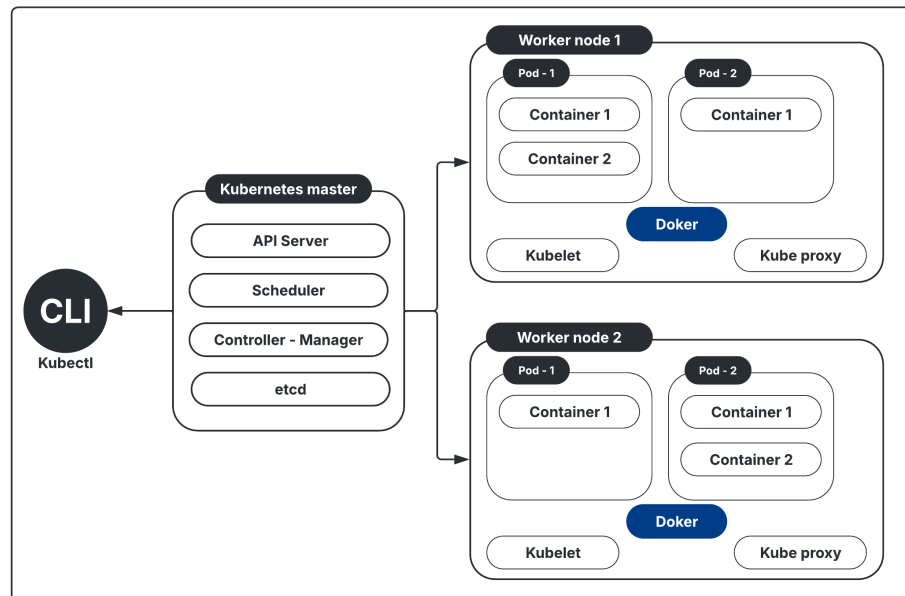


FIGURE 9 – Orchestrateur des conteneurs

- **Scheduler** : responsable de l’affectation des pods aux nœuds workers selon les ressources disponibles ;
- **Controller Manager** : Supervise l’état du cluster et maintient la configuration désirée ;
- **etcd** : Base de données distribuée stockant toutes les données de configuration du cluster.

Les nœuds **workers** hébergent et exécutent les applications conteneurisées via plusieurs composants :

- **Pods** : unités d’exécution contenant un ou plusieurs conteneurs partageant le même contexte ;
- **Container Runtime Interface (CRI)** : Runtime de conteneurisation gérant l’exécution des conteneurs (Docker) ;
- **Kubelet** : Agent Kubernetes communiquant avec le maître et gérant les pods localement ;
- **Kube-proxy** : Proxy réseau gérant les règles de trafic et l’équilibrage de charge.

Le **kubectl** est le CLI permettant aux administrateurs d’interagir avec le cluster via l’API Server.

I.6. Généralités sur la transformée de Fourier (TF)

Le traitement du signal est une discipline technique qui utilise les ressources de l'électronique, de l'informatique et de la physique appliquée pour développer et interpréter les signaux. Il permet de concevoir, d'examiner et de modifier les signaux afin de les exploiter. Son domaine d'application englobe donc tous les secteurs liés à la perception, à la transmission ou à l'utilisation des informations transportées par ces signaux.

I.6.1. Notions fondamentales sur les signaux

Un signal est une fonction mathématique qui représente la variation d'une grandeur physique en fonction d'une ou plusieurs variables indépendantes, généralement le temps ou l'espace. Il constitue le support de l'information dans les systèmes techniques. En traitement du signal, l'analyse des signaux permet d'en extraire des caractéristiques utiles à diverses applications, qu'il s'agisse de reconnaissance vocale, de détection d'anomalies, ou de transmission d'informations. Le bruit peut être décrit comme tout phénomène perturbateur qui entrave la perception ou l'interprétation d'un signal.

On distingue plusieurs types de signaux, principalement :

- Les **signaux continus** : il est continu et, en théorie, peut posséder un nombre illimité de valeurs dans une certaine fourchette. on les appelle aussi *signaux analogiques* (figure 10); Ils sont généralement issus de processus physiques et sont modélisés par des fonctions mathématiques continues définies sur un domaine temporel ou spatial continu;

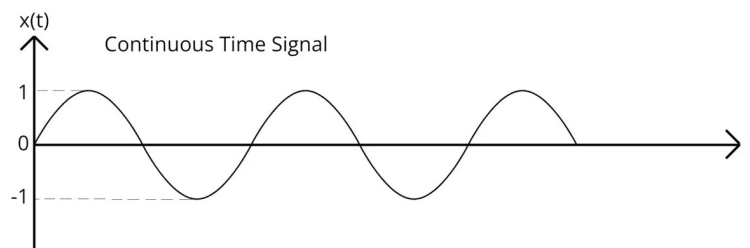


FIGURE 10 – Signal continue

- Les **signaux discrets** : ils sont définis uniquement à des instants précis et discrets, contrairement aux signaux continus. On les appelle aussi *signaux numériques* lorsqu'ils sont également quantifiés en amplitude (figure 11); Ils résultent généralement de l'échantillonnage de signaux analogiques ou proviennent directement de systèmes numériques, et sont modélisés par des

séquences mathématiques discrètes $x[n]$ où n représente l'indice temporel discret ;

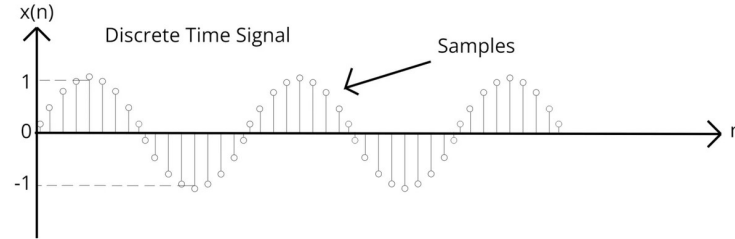


FIGURE 11 – Signal discret

La transformée de Fourier est une opération mathématique qui permet de représenter un signal dans le domaine fréquentiel. Elle transforme un signal $f(t)$ du domaine temporel vers une fonction $\hat{f}(v)$ exprimant l'amplitude de ses composantes sinusoïdales. Cette représentation est fondamentale pour analyser les fréquences contenues dans un signal, détecter des motifs, compresser des données ou concevoir des filtres. Elle est omniprésente dans les domaines du traitement du signal, de l'audio, des communications et de la vision.

La transformée de Fourier (TF) d'un signal défini par une fonction $f(t)$ intégrable, notée $\mathcal{F}[f]$ ou $\hat{f}(v)$, est définie par :

$$\hat{f}(v) = \mathcal{F}[f(t)] = \int_{-\infty}^{+\infty} f(t) e^{-2i\pi vt} dt \quad (\text{I.5})$$

où v représente la fréquence en Hz et i l'unité imaginaire $i = \sqrt{-1}$.

La TF inverse, notée $\mathcal{F}^{-1}$ est également définie et permet, à partir de la TF de reconstruire la fonction $f(t)$. Elle s'écrit :

$$f(t) = \mathcal{F}^{-1}[\hat{f}](t) = \int_{-\infty}^{+\infty} \hat{f}(v) e^{2i\pi vt} dv. \quad (\text{I.6})$$

I.6.2. Traitement des signaux numériques

I.6.2.1. Transformée de Fourier Discrète

Le calcul de la transformée de Fourier présente un défi lorsqu'il s'agit de signaux discrets, ce qui est généralement le cas puisque l'acquisition d'un signal nécessite un échantillonnage. Par conséquent, ce calcul est réalisé en utilisant l'algorithme de la Transformée de Fourier Discrète (TFD), dont une version plus rapide, la FFT (Fast Fourier Transform), a été proposée par Cooley et Tukey en 1965 [11].

La Transformée de Fourier Discrète (TFD) est l'outil utilisé pour analyser le contenu fréquentiel d'un signal discret et de durée finie. Elle transforme une séquence de N échantillons dans le domaine temporel en une séquence de N composantes dans le domaine fréquentiel.

La TFD d'un signal $x[n]$ avec $n = \{0, 1, \dots, N-1\}$, encore appelé *spectre* du signal notée $X[k]$ est donnée par la formule :

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-i \frac{2\pi}{N} kn} \quad \text{avec} \quad k = \{0, 1, \dots, N-1\} \quad (\text{I.7})$$

Le signal $x[n]$ peut être retrouvé à partir de son spectre $X[k]$ en utilisant la **transformée de Fourier discrète inverse** donnée par :

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{i \frac{2\pi}{N} kn} \quad \text{avec} \quad n = \{0, 1, \dots, N-1\} \quad (\text{I.8})$$

I.6.2.2. Transformée de Fourier Rapide (FFT)

La FFT est un algorithme efficace pour calculer la TFD qui réduit la complexité de $O(N^2)$ à $O(N \log N)$. L'algorithme le plus courant est celui de Cooley-Tukey, qui utilise une approche "diviser pour régner".

Le principe de base consiste à décomposer une TFD de taille N en deux TFD de taille $N/2$:

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, \quad (\text{I.9})$$

où $W_N = e^{-i2\pi/N}$ est le facteur de rotation.

En séparant les indices pairs et impairs :

$$X[k] = \sum_{m=0}^{N/2-1} x[2m] W_N^{k(2m)} + \sum_{m=0}^{N/2-1} x[2m+1] W_N^{k(2m+1)}. \quad (\text{I.10})$$

Nous avons pour le terme pair :

$$W_N^{k(2m)} = \left(e^{-i \frac{2\pi}{N}} \right)^{k(2m)} = e^{-i \frac{4\pi km}{N}} = e^{-i \frac{2\pi km}{N/2}} = W_{N/2}^{km}.$$

Pour le terme impair :

$$W_N^{k(2m+1)} = W_N^{2km+k} = W_N^{2km} \cdot W_N^k = (W_N^2)^{km} \cdot W_N^k = W_{N/2}^{km} \cdot W_N^k.$$

En remplaçant ces deux résultats dans la somme :

$$X[k] = \sum_{m=0}^{N/2-1} x[2m] W_{N/2}^{km} + W_N^k \sum_{m=0}^{N/2-1} x[2m+1] W_{N/2}^{km} \quad (\text{I.11})$$

$$= E[k] + W_N^k \cdot O[k], \quad (\text{I.12})$$

où $E[k]$ et $O[k]$ sont respectivement les TFD des échantillons pairs et impairs.

Périodicité du facteur de rotation

Par définition,

$$W_N^{k+N/2} = \left(e^{-i\frac{2\pi}{N}}\right)^{k+N/2} = e^{-i\frac{2\pi}{N}(k+N/2)} = e^{-i\frac{2\pi k}{N}} \cdot e^{-i\pi}.$$

Or $e^{-i\pi} = -1$, donc :

$$W_N^{k+N/2} = -W_N^k.$$

—

À partir de la décomposition précédente, on a pour l'indice $k + N/2$:

$$X\left[k + \frac{N}{2}\right] = E\left[k + \frac{N}{2}\right] + W_N^{k+\frac{N}{2}} \cdot O\left[k + \frac{N}{2}\right]. \quad (\text{I.13})$$

1. Simplification de $E\left[k + \frac{N}{2}\right]$:

$$E\left[k + \frac{N}{2}\right] = \sum_{m=0}^{N/2-1} x[2m] W_{N/2}^{\left(k+\frac{N}{2}\right)m} \quad (\text{I.14})$$

$$= \sum_{m=0}^{N/2-1} x[2m] \left(W_{N/2}^{km} \cdot W_{N/2}^{\frac{N}{2}m}\right). \quad (\text{I.15})$$

Or $W_{N/2}^{\frac{N}{2}m} = \left(e^{-i\frac{4\pi}{N}}\right)^{\frac{N}{2}m} = e^{-i2\pi m} = 1$ car m est entier. Donc :

$$E\left[k + \frac{N}{2}\right] = E[k].$$

2. Même simplification pour $O[k + N/2]$:

$$O\left[k + \frac{N}{2}\right] = O[k].$$

3. En utilisant la relation $W_N^{k+N/2} = -W_N^k$, on obtient alors :

$$X\left[k + \frac{N}{2}\right] = E[k] + (-W_N^k) \cdot O[k] \quad (\text{I.16})$$

$$= E[k] - W_N^k \cdot O[k]. \quad (\text{I.17})$$

—

Ainsi, les deux résultats finaux sont :

$$X[k] = E[k] + W_N^k \cdot O[k], \quad (\text{I.18})$$

$$X\left[k + \frac{N}{2}\right] = E[k] - W_N^k \cdot O[k]. \quad (\text{I.19})$$

Cette relation de symétrie est au cœur de l'algorithme FFT : elle permet de calculer les coefficients pour k et $k + N/2$ à partir des mêmes sous-transformées $E[k]$ et $O[k]$, réduisant le nombre d'opérations.

Le pseudo-code suivant illustre l'implémentation récursive classique de l'algorithme FFT (Algo 1) :

Algorithm 1: Algorithme FFT

Input: Un vecteur x de taille N , avec $N = 2^m$ (puissance de 2)

Output: Le vecteur transformé X contenant les composantes fréquentielles

```

1 Function  $\text{FFT}(x)$  :
2    $N \leftarrow \text{longueur}(x)$ ;
3   if  $N = 1$  then
4     return  $x$ ; // cas de base
    ▷ Diviser le signal en sous-signaux pairs et impairs
5    $x_{\text{pair}} \leftarrow x[0], x[2], x[4], \dots, x[N-2]$ ;
6    $x_{\text{impair}} \leftarrow x[1], x[3], x[5], \dots, x[N-1]$ ;
    ▷ Appels récursifs
7    $X_{\text{pair}} \leftarrow \text{FFT}(x_{\text{pair}})$ ;
8    $X_{\text{impair}} \leftarrow \text{FFT}(x_{\text{impair}})$ ;
    ▷ Combiner les résultats
9   for  $k \leftarrow 0$  to  $N/2 - 1$  do
10     $W \leftarrow e^{-2\pi i k / N}$ ; // facteur de rotation
11     $X[k] \leftarrow X_{\text{pair}}[k] + W \cdot X_{\text{impair}}[k]$ ;
12     $X[k + N/2] \leftarrow X_{\text{pair}}[k] - W \cdot X_{\text{impair}}[k]$ ;
13  return  $X$ ;

```

L'algorithme divise une TFD de taille N en deux TFD de taille $N/2$, avec un travail supplémentaire de $O(N)$ pour combiner les résultats (opérations *butterfly*). La relation de récurrence est :

$$T(N) = 2T\left(\frac{N}{2}\right) + O\left(\frac{N}{2}\right)$$

En résolvant cette récurrence, on obtient :

$$T(N) = O(N \log N)$$

Cette complexité temporelle $O(N \log N)$ rend la FFT particulièrement adaptée au traitement de signaux de grande taille, offrant une accélération significative par rapport à la TFD classique, surtout pour des grandes valeurs de N .

I.6.3. Quelques domaines d'application de la TF

L'efficacité de la FFT élargit considérablement le champ d'application de la transformée de Fourier, en permettant l'analyse fréquentielle rapide de signaux volumineux. Voici les principaux domaines où la transformée de Fourier, optimisée par la FFT, est utilisée :

- **Traitement du signal** : Analyse des fréquences dans les signaux audio, filtrage numérique, détection de motifs, et compression de données ;
- **Télécommunications** : Modulation et démodulation des signaux (par exemple, dans les réseaux 4G/5G), analyse de la bande passante, et réduction du bruit ;
- **Imagerie et vision par ordinateur** : Traitement d'images pour la compression (JPEG), le filtrage, l'analyse de textures, et la reconstruction en tomographie (IRM, scanner ;
- **Acoustique et audio** : Analyse spectrale pour l'égalisation, la reconnaissance vocale, et la synthèse sonore ;
- **Analyse des signaux** : La FFT est utilisée pour analyser les signaux d'électrocardiogramme (ECG) et d'électroencéphalogramme (EEG), facilitant ainsi le diagnostic de maladies telles que l'arythmie et les troubles du sommeil ;
- **Informatique et cryptographie** : Utilisation dans les algorithmes de multiplication rapide de polynômes et l'analyse spectrale pour la sécurité des données.

Les applications de la transformée de Fourier englobent aujourd'hui un vaste éventail de domaines, illustrant sa polyvalence et son efficacité dans la résolution de problèmes complexes en traitement du signal, télécommunications, imagerie, et bien au-delà.

I.7. Conclusion

Dans ce premier chapitre, nous avons posé les fondements conceptuels et technologiques nécessaires à la compréhension du travail présenté dans ce mémoire. Nous avons tout d'abord retracé l'évolution des paradigmes de développement logiciel, depuis les approches procédurales jusqu'aux architectures orientées services et microservices, qui constituent aujourd'hui les bases de nombreux systèmes distribués modernes.

Nous avons ensuite examiné les différentes formes de composition des services, qu'elles soient statiques ou dynamiques, tout en soulignant les limites de ces approches dans des environnements fortement distribués, évolutifs et réactifs. Cette

analyse a permis de motiver l'intérêt croissant pour la composition incrémentale, qui sera explorée plus en profondeur dans le chapitre suivant.

Parallèlement, ce chapitre a permis d'introduire les infrastructures techniques sur lesquelles reposent les architectures cloud-native actuelles : la virtualisation, la conteneurisation, et leur orchestration via des plateformes comme Kubernetes.

Enfin, nous avons présenté les notions fondamentales liées aux signaux numériques et à la transformée de Fourier, en insistant sur les propriétés structurelles de l'algorithme FFT qui en font un excellent candidat pour un benchmark de composition de services. La granularité, le parallélisme naturel et la structure récursive de la FFT en font un cas d'étude particulièrement adapté à une exécution paresseuse et guidée par les données.

Le cadre ainsi posé nous permet, dans le chapitre suivant, d'examiner l'état de l'art des approches de composition incrémentale, en mettant en lumière les modèles formels qui la sous-tendent, les travaux existants et les systèmes expérimentaux déjà proposés.

II

CHAPITRE

ÉTAT DE L'ART SUR LA COMPOSITION INCRÉMENTALE DES SERVICES

SOMMAIRE

II.1 - Introduction	39
II.2 - Description des Services Web	40
II.3 - Taxonomie sur la composition de services	41
II.4 - Calcul incrémental et exécution paresseuse	44
II.5 - Conclusion	51

II.1. Introduction

Après avoir établi les bases conceptuelles et techniques relatives aux architectures orientées services, aux infrastructures cloud-native et à la transformée de Fourier, ce chapitre explore l'évolution des approches de composition de services, depuis les modèles traditionnels vers les paradigmes incrémentaux.

Nous commencerons par situer cette approche dans l'évolution des modèles de composition, en montrant comment les besoins en flexibilité, en scalabilité et en réactivité dans les systèmes modernes ont conduit à dépasser les paradigmes traditionnels. Ensuite, nous présenterons les principes fondamentaux de la composition incrémentale : exécution pilotée par les données, granularité des traitements, activation paresseuse et suivi dynamique des dépendances.

Enfin, nous passerons en revue les travaux de recherche et prototypes existants qui exploitent ce paradigme, en mettant en évidence leurs apports, leurs limites et les perspectives ouvertes pour des cas d'application concrets comme celui étudié dans ce mémoire.

II.2. Description des Services Web

Les efforts de recherche et de développement récents autour des services Web ont conduit à un certain nombre de spécifications qui définissent aujourd'hui les architectures de référence des services Web. Pour garantir l'interopérabilité dans une SOA, des propositions de standards ont été élaborées. Nous citons, notamment les standards émergents SOAP, WSDL, UDDI et REST.

- **SOAP** : est un protocole de messagerie créé en 1998 pour l'implémentation de services Web [45]. Il facilite la communication entre applications et les appels RPC en utilisant des messages XML structurés dans des enveloppes SOAP. Ces enveloppes contiennent un en-tête optionnel pour les métadonnées (authentification, routage) et un corps obligatoire pour les données principales. SOAP s'appuie sur les protocoles de transport existants comme HTTP et SMTP plutôt que de créer son propre protocole de communication ;
- **WSDL** est un standard utilisé pour décrire les services SOAP de manière indépendante de leur implémentation. Il définit l'interface du service en séparant la description abstraite de la fonctionnalité des détails concrets d'accès. La partie abstraite décrit le service à travers les messages échangés et inclut des éléments "*operation*" regroupés dans un élément "*interface*", sans spécifier le protocole. La partie concrète établit la liaison entre la définition d'interface et un protocole spécifique. WSDL fournit ainsi tous les détails nécessaires pour localiser le service et comprendre l'ensemble des opérations qu'il prend en charge ;
- **UDDI** est un système de registres centralisés qui facilite la découverte des services Web en fonctionnant comme un annuaire numérique [1]. Il organise les informations en trois catégories : pages blanches (noms et contacts), pages jaunes (catégorisation par types de services), et pages vertes (détails techniques). UDDI permet aux utilisateurs de rechercher et localiser systématiquement des fournisseurs de services Web selon leurs caractéristiques spécifiques.
- **REST** est un style architectural basé sur le concept de ressources identifiées par des URI et manipulées via une interface uniforme utilisant les verbes HTTP (*GET*, *POST*, *PUT*, *DELETE*) [45]. Il s'appuie sur HTTP comme protocole de transport et n'impose aucun format d'échange spécifique, permettant l'utilisation de XML, JSON ou autres formats. Les applications res-

pectant pleinement ces critères sont appelées *RESTful* et permettent de développer des architectures orientées ressources.

II.3. Taxonomie sur la composition de services

La composition des services est un concept central dans les architectures orientées services (SOA), qui permet de combiner des services distincts pour créer des applications ou des processus plus complexes. Dans un monde où l'agilité, la réutilisabilité et l'interopérabilité sont des enjeux majeurs, la capacité à assembler efficacement des services devient essentielle. Cependant, cette composition n'est pas un processus unifié, mais se décline en plusieurs approches et stratégies, souvent influencées par les objectifs techniques, les contraintes métier, ou même les évolutions des services eux-mêmes.

Composition statique des services

Orchestration. L'orchestration des services s'appuie sur divers langages de définition. Cette section détaille les langages les plus employés dans ce domaine :

- **XLANG** est une extension WSDL développée par Microsoft, qui offre un framework d'orchestration de services web et de définition d'accords collaboratifs, basé sur des fondements théoriques de calcul. Les actions constituent ses éléments fondamentaux, intégrant les quatre opérations WSDL standard (requête/réponse, sollicitation, notification) et ajoutant deux catégories spécifiques : *arrêts* (date-limite et durée) et *exceptions* ;
- **WSFL** Web Services Flow Language [30] constitue un langage XML destiné à spécifier les services web composites. Il distingue deux approches compositionnelles : premièrement, la modélisation d'usage coordonné d'un ensemble de services web pour atteindre un objectif métier spécifique, générant ainsi une spécification de processus d'affaires ; deuxièmement, la définition des patterns d'interaction entre services web, produisant une description des échanges globaux associés ;
- **BPEL4WS** [25] (également désigné BPEL ou BPELWS) constitue un langage XML spécialisé dans la spécification des processus d'affaires. Son architecture permet l'échange et la modification de données distribuées entre organismes multiples via l'orchestration de services web. Ce langage modélise les interactions de processus métier fondés sur les services web, tant en contexte intra-entreprise qu'inter-organisationnel. Les organisations adoptant BPEL peuvent établir leurs workflows métier tout en assurant l'interopérabilité à l'échelle corporative et avec leurs partenaires commerciaux

dans un écosystème de services web. La spécification BPEL unifie et supprime les standards WSFL et XLANG.

Chorégraphie. Contrairement à l'orchestration centralisée, la chorégraphie adopte une stratégie décentralisée et coopérative pour assembler les services. Les langages de chorégraphie majeurs comprennent :

- **WSCL** [5] : propose une description XML des services web centrée sur leurs échanges conversationnels et la prise en compte des messages transmis. Conçu pour un déploiement complémentaire avec WSDL, WSCL exploite les définitions WSDL pour spécifier les opérations disponibles et leur chorégraphie, tandis que WSDL fournit les implémentations concrètes des définitions de messages et les spécifications techniques des éléments manipulés ;
- **WSCI** [53] : constitue un langage XML axé sur la modélisation des services web comme interfaces définissant les flux de messages échangés (chorégraphie des messages). Il vise à caractériser le comportement externe observable des services en exprimant les dépendances logiques et temporelles inter-messages via des mécanismes de contrôle séquentiel, de corrélation, de gestion d'erreurs et de transactions. WSCI réutilise les définitions abstraites WSDL pour spécifier ultérieurement les modalités d'implémentation concrète des éléments manipulés dans la modélisation de services.

Composition dynamique

Approches orientées Intelligence Artificielle. Un service web peut être spécifié à l'aide de ses conditions et effet qui sont des concepts très proches de ceux de la planification.

Plusieurs branches de l'IA intègrent le processus de composition ; nous notons notamment :

- **Composition à base de règles** : cette approche de planification s'appuie sur un ensemble de règles prédéfinies et de conditions dans la même philosophie que les déclarations *if-then-else* ;
- **Composition avec SMA** [29] : l'implémentation de la composition de services peut exploiter les SMA. Chaque agent représente un service et traite une portion de la requête utilisateur selon ses capacités spécifiques ;
- **Calcul situationnel** : cette méthode modélise les requêtes utilisateur et contraintes de services sous forme de prédicats du premier ordre dans le langage de calcul situationnel. Les services sont formalisés comme actions (primitives ou complexes) dans ce même langage. Des modèles sont ensuite générés via

des règles de déduction et contraintes, puis instanciés à l'exécution selon les préférences utilisateur ;

- **Composition par planification** [39] :cette approche organise les services web selon leurs préconditions et effets. Diverses méthodes ont été développées pour traiter la composition servicielle comme un problème de planification, notamment l'approche PDDL qui vise à standardiser les données d'entrée des planificateurs. Ce langage propose un formalisme unifié pour décrire les domaines de planification, incitant ainsi la communauté de recherche en services web à l'adapter au contexte compositionnel.

Approches Formelles. Les modèles sémantiques formels apparaissent comme l'une des meilleures solutions pour spécifier la composition des services web.

Plusieurs formalismes ont été développés dans cette perspective, incluant les automates, diagrammes d'activités, algèbres de processus et réseaux de Pétri.

- **Automate** ces modèles constituent des outils largement employés pour la spécification formelle de systèmes complexes. Un automate comprend un ensemble de nœuds représentant les états et des arcs étiquetés décrivant les transitions inter-états. Dans le contexte de la modélisation compositionnelle de services, chaque appel de service correspond à un état spécifique, un état distinct signale la terminaison d'exécution du service, et chaque réponse génère un événement associé ;
- **Diagrammes UML** [20] UML constitue désormais un standard reconnu pour la modélisation orientée objet de systèmes complexes. Les diagrammes UML se répartissent en trois catégories : (i) diagrammes structurels modélisant les aspects statiques (diagrammes de *classes*, d'*objets*), (ii) diagrammes comportementaux (diagrammes d'*activités*, d'*états-transitions*), et (iii) les diagrammes d'interactions dynamiques (les diagrammes de *séquences*). Le *diagramme d'activité* s'avère particulièrement prisé pour la composition de services web. Dans cette approche, l'exécution de services est généralement représentée par une action, la transition entrante modélise l'invocation du service tandis que la transition sortante signale la fin d'exécution ;
- **Algèbre de Processus** [32] ces familles de langages formels permettent la modélisation de systèmes complexes, notamment les architectures distribuées et concurrentielles ;
- **Réseaux de Pétri** [10] les RdPs les RdPs constituent un outil de modélisation pour les systèmes dynamiques à événements discrets. Leur principal atout réside dans leur représentation graphique facilitant considérablement la modélisation conceptuelle de systèmes variés. Les RdPs jouissent d'une

forte reconnaissance dans le domaine de la gestion des processus métiers (BPM) grâce à leur capacité à modéliser une grande diversité de processus et de flux de contrôle.

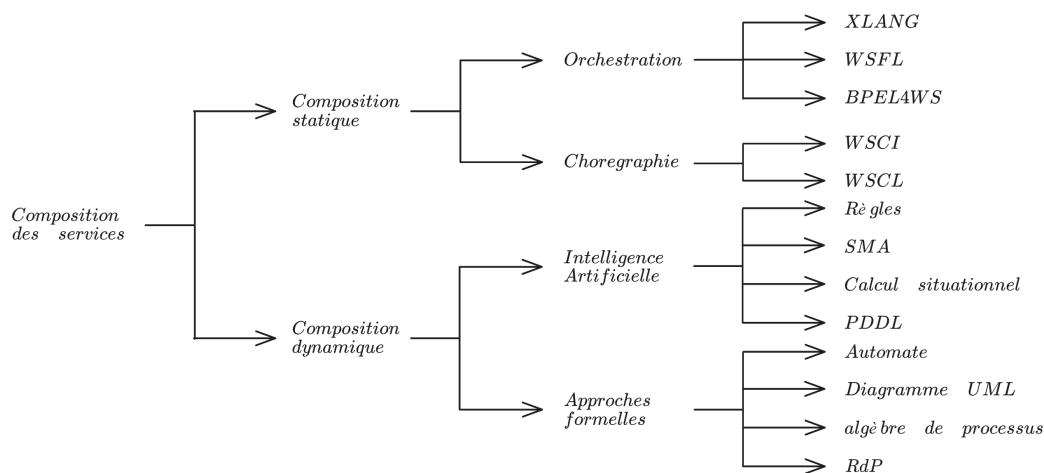


FIGURE 12 – Taxonomie des approches de composition des services

La figure 12 présente une taxonomie hiérarchique de la composition de services structurée en deux approches principales : la composition statique et la composition dynamique. La composition statique se subdivise en orchestration (coordination centralisée utilisant XLANG, WSFL, BPELAWS) et chorégraphie (coordination décentralisée avec WSCI, WSCL), offrant des processus prédéfinis et stables. La composition dynamique comprend les approches d'intelligence artificielle (règles, SMA, calcul situationnel, PDDL) et les approches formelles (automates, diagrammes UML, algèbre de processus, RdP), permettant une adaptation en temps réel. Cette classification illustre l'évolution des paradigmes de composition depuis les approches statiques traditionnelles vers des solutions dynamiques plus flexibles, chaque catégorie répondant à des besoins spécifiques en termes de prévisibilité, contrôle, adaptabilité et vérification formelle dans les architectures orientées services.

II.4. Calcul incrémental et exécution paresseuse

Dans les environnements modernes, où les services sont hautement distribués et les données souvent partielles ou disponibles de façon asynchrone, il devient indispensable de concevoir des mécanismes de composition qui s'adaptent dynamiquement à l'état du système. Contrairement aux modèles traditionnels, qui supposent une orchestration globale et rigide, le calcul incrémental et l'exécution

paresseuse offrent une alternative plus souple, réactive et localement autonome. Ces paradigmes permettent de déclencher uniquement les services nécessaires, au moment où leurs dépendances sont satisfaites, réduisant ainsi la consommation de ressources tout en augmentant la résilience et la scalabilité.

II.4.1. Les GAG pour la collaboration distribuée

Éric Badouel et al. [4] ont proposé dans leur article *"Active Workspaces : Distributed Collaborative Systems based on Guarded Attribute Grammars"* une nouvelle approche pour la conception de systèmes collaboratifs distribués, en introduisant le modèle des Active Workspaces (AW), combiné au formalisme des Grammaires Attribuées Gardées (GAG). Le contexte de cette contribution est celui des systèmes de gestion de processus métier, traditionnellement fondés sur des modèles de workflow centralisés. Ces derniers orchestrent les tâches à accomplir mais présentent plusieurs limitations majeures : les données sont souvent négligées dans la logique du processus, et les parties prenantes humaines sont considérées comme de simples ressources passives. De plus, la rigidité et la centralisation de ces modèles les rendent peu adaptés à des environnements dynamiques et distribués.

Pour remédier à cela, les auteurs s'appuient sur le concept d'espace de travail actif (AW), une évolution des cartes mentales, qui permet à chaque utilisateur d'organiser ses tâches et de collaborer de manière asynchrone avec d'autres. Le problème que l'article adresse est donc celui de la modélisation cohérente et distribuée de la collaboration, sans orchestration centrale, en intégrant à la fois la structure des données, les règles de transformation, et la participation active des utilisateurs.

Méthodologie. Les auteurs ont défini un Active Workspace comme un arbre structuré personnel à chaque utilisateur, représentant ses tâches, enrichi par des attributs de données qui guident l'exécution. Ce modèle s'inspire des cartes mentales, avec une orientation vers l'interactivité et la collaboration distribuée.

Les GAG sont utilisées pour formaliser les transformations de l'arbre : chaque règle de production exprime comment une tâche (nœud) peut être décomposée en sous-tâches, sous conditions de garde (conditions sur les données disponibles).

L'exécution est répartie entre les utilisateurs. Chacun dispose d'une vue locale de l'artefact partagé et peut appliquer des règles de manière autonome, sans coordination centrale, dès que les données locales satisfont les conditions.

Les changements locaux sont diffusés via un système de messagerie asynchrone. Lorsqu'un utilisateur modifie un nœud, ses effets sont propagés aux autres parties concernées du système, assurant une cohérence distribuée entre les workspaces.

Plutôt que d'orchestrer les tâches à l'aide d'un moteur central, les auteurs ont laissé aux GAG le soin d'implémenter implicitement la coordination, en encodant dans les gardes les conditions nécessaires pour que les règles soient applicables. Ainsi, la progression des tâches s'adapte naturellement à l'état courant des données.

Résultats. Pour évaluer leur approche, les auteurs ont illustré leur modèle à travers un système de surveillance des maladies. Ce cas pratique consiste en un processus distribué où plusieurs acteurs, tels que des médecins, des biologistes, et des épidémiologistes, collaborent pour suivre et analyser des cas suspects de maladies infectieuses.

Dans ce contexte, chaque acteur, dispose d'un espace de travail actif représenté par une carte structurée en arbre, où chaque nœud correspond à une tâche ou une information (figure 13). Lorsqu'une tâche doit être effectuée, un nœud ouvert est créé, décrivant le travail à faire. La communication entre acteurs se fait par messages asynchrones, permettant une gestion distribuée. Ce modèle permet de décomposer le processus, d'assurer l'automatisation des transferts d'informations, et de garantir la flexibilité du système. Il montre que la modélisation par GAG facilite la coordination distribuée tout en conservant modularité, cohérence et adaptabilité.

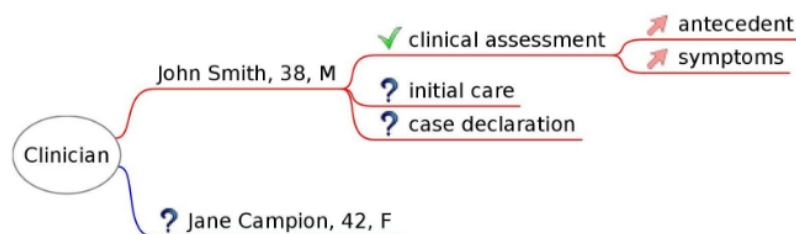


FIGURE 13 – Espace de travail actif d'un clinicien [4]

En résumé, leur solution a permis de simuler un processus collaboratif multi-acteurs où chaque partie peut évoluer indépendamment, tout en étant cohérente avec l'ensemble grâce aux règles formelles. Cela a démontré que la modélisation par GAG est adaptée pour gérer efficacement des systèmes collaboratifs distribués, flexibles et réactifs, permettant une meilleure coordination dans des environnements décentralisés et potentiellement dégradés.

II.4.2. Processus d'entreprises distribués et disponibles via GAG et Publish/Subscribe

Maurice Tchoupé et Joskel Tagueu [51] ont proposé dans leur article *"Guarded Attribute Grammars and Publish/Subscribe for implementing distributed collaborative business processes with high data availability"* un système collabora-

tif distribué utilisant des Grammaires Attribuées Gardées (GAG) et un protocole de publish/subscribe avec redirection de souscriptions (PubSub-RS) pour améliorer la disponibilité des données et faciliter la communication asynchrone entre les acteurs, améliorant ainsi la collaboration dans les processus métier sur différents sites géographiques.

Méthodologie. La méthodologie proposée dans l'article se divise en plusieurs étapes clés pour implémenter des processus métiers distribués utilisant GAG et le protocole pub/sub-RS. Tout d'abord, à partir d'une description textuelle du processus, les acteurs sont identifiés, et leurs GAG locaux sont spécifiés. Ensuite, un moteur GAG est développé pour exécuter ces processus en environnement distribué. La communication entre acteurs s'appuie sur le protocole pub/sub-RS, qui assure la transmission incrémentale et en temps réel des données semi-structurées, même partiellement produites, par le biais de la redirection des souscriptions.

Une architecture basée sur des composants est également proposée pour déployer ces processus, permettant une exécution distribuée efficace et flexible. La méthodologie inclut ainsi la formalisation des processus, la mise en œuvre des GAG locaux, le développement d'un moteur d'exécution, et la configuration d'un système de communication pub/sub-RS permettant la synchronisation et la collaboration entre acteurs en temps réel.

Résultats. Pour évaluer leur approche, les auteurs ont illustré leur solution à travers le processus de soumission de thèse à l'université de Dschang. Dans ce contexte, plusieurs acteurs, tels que l'étudiant, le département, l'école doctorale et le rectorat, participent de manière distribuée à différentes étapes du traitement du dossier. La solution modélisée avec GAG et communiquant via le protocole pub/sub-RS permet de suivre en temps réel chaque étape du processus. Lorsqu'une étape est effectuée, comme l'approbation par le rectorat, l'information est immédiatement transmise à tous les acteurs concernés, même si les données sont produites de façon collaborative et incrémentale par plusieurs acteurs. Cette capacité assure que chaque acteur, en particulier l'étudiant, est rapidement informé des avancées ou des rejets de son dossier, facilitant une gestion plus dynamique et une prise de décision plus rapide tout au long du processus. Ces résultats montrent que l'approche permet d'assurer une haute disponibilité et une synchronisation efficace des données dans un contexte distribué et collaboratif.

II.4.3. Vers une exécution incrémentale dans les architectures orientées services

II.4.3.1. Architecture orientée services basée sur les calculs incrémentaux

Joskel Tagueu et al. [49] ont proposé dans leur article "*Lazy Services : A Service Oriented Architecture based on Incremental Computations and Commitments*" une architecture orientée services novatrice basée sur le calcul incrémental et le concept de **services paresseux**. Leur objectif est de répondre aux limites des modèles classiques de composition de services, en proposant une exécution réactive, progressive et pilotée par les données, adaptée aux environnements distribués dynamiques. Les systèmes orientés services classiques supposent généralement que toutes les données nécessaires à l'exécution d'un service sont disponibles dès son déclenchement (approche *tout ou rien*). Cette hypothèse est difficile à maintenir dans des contextes où les données arrivent progressivement, de manière asynchrone ou partielle — comme c'est souvent le cas dans les environnements cloud-native, les systèmes IoT, ou les workflows scientifiques distribués. L'objectif de l'article est de formaliser une exécution *lazy* (paresseuse), dans laquelle un service est activé au fur et à mesure que ses dépendances sont satisfaites, tout en permettant une production incrémentale de ses résultats.

Méthodologie. L'architecture proposée repose sur plusieurs concepts clés :

- **Services paresseux** : un service est défini par un ensemble d'engagements sur les données qu'il produira, sans attendre que l'ensemble de ses entrées soit complètement disponible ;
- **Utilisation des Grammaires Attribuées Gardées (GAG)** : les services sont formalisés comme des règles de production dont les gardes expriment les conditions d'activation. Ces règles permettent d'exécuter les services partiellement, dès qu'une portion de données est disponible ;
- **Propagation incrémentale** : chaque production partielle de données peut déclencher d'autres services en aval, créant ainsi une exécution réactive et distribuée, sans plan d'ordonnancement global ;
- **Engagements** : les services publient à l'avance les résultats qu'ils s'engagent à produire, ce qui permet aux services consommateurs de s'organiser sans attendre leur production effective.

Résultats. Les auteurs ont validé leur approche par une expérimentation portant sur des processus de composition de services scientifiques. Le scénario testé montre que :

- les services peuvent être déclenchés progressivement et en parallèle dès qu'une partie de leurs dépendances est disponible ;
- la stratégie paresseuse réduit le temps global d'exécution dans un contexte de données partiellement disponibles ;
- l'exécution devient naturellement tolérante aux défaillances ponctuelles, car la progression du processus dépend uniquement des données locales disponibles et non d'une orchestration centrale.

Cette architecture a montré une capacité élevée à gérer des dépendances complexes et à offrir une meilleure scalabilité dans des environnements distribués.

II.4.3.2. Moteur de composition incrémentale

Joskel Tagueu et al. [48] ont présenté dans leur article "*A Service Composition Engine for Incremental Computation*" une architecture orientée services reposant sur un moteur d'exécution distribué basé sur les **Grammaires Attribuées Gardées (GAG)**. L'objectif de ce moteur est de permettre une exécution réactive, flexible et incrémentale des services dans un environnement cloud-native, en s'appuyant sur des composants conteneurisés et orchestrés par Kubernetes.

Contexte et objectif. Les architectures modernes exigent des systèmes capables de gérer dynamiquement des services distribués, dont les données peuvent être disponibles de manière partielle ou asynchrone. Les moteurs de composition classiques sont souvent limités par une orchestration rigide et centralisée, qui ne s'adapte pas facilement à la nature dynamique des environnements cloud, edge, ou orientés flux. Le moteur de composition proposé vise à offrir une exécution autonome, résiliente et hautement distribuée, où chaque service est activé localement selon l'état de ses dépendances de données.

Méthodologie. Le moteur repose sur une infrastructure modulaire, articulée autour de plusieurs technologies :

- **Déclaration des services :** Chaque service est spécifié sous forme d'une règle GAG dans un fichier YAML, décrivant ses entrées, sorties et conditions d'activation ;
- **Fonctions locales :** Les terminaux (fonctions de traitement) sont codés en Java et associés aux règles de production ;
- **Conteneurisation :** L'ensemble des composants est embarqué dans des conteneurs Docker. Un compilateur assemble les spécifications, les fonctions Java et un moteur GAG dans un artefact .jar prêt à être exécuté ;

- **Déploiement distribué** : Les conteneurs sont orchestrés sur un cluster Kubernetes. Chaque conteneur expose une API REST et communique via des messages asynchrones de type publish/subscribe ;
- **Exécution incrémentale** : Le moteur exécute les règles dès que leurs gardes sont satisfaites localement. Chaque production partielle de données peut activer en cascade d'autres services, garantissant une exécution fluide, parallèle et adaptative.

Cette méthodologie permet de découpler entièrement la spécification fonctionnelle du service de son déploiement, tout en assurant une exécution pilotée par les données disponibles.

Résultats. Les auteurs ont validé le fonctionnement du moteur en l'appliquant à l'exécution distribuée de l'algorithme de tri par fusion (merge sort). L'approche démontre plusieurs bénéfices :

- **Parallélisme naturel** : Chaque sous-composant du tri (division et fusion) est traité indépendamment dès que ses données sont disponibles ;
- **Réactivité** : L'exécution se déclenche localement sans attendre la disponibilité complète du processus global.
- **Tolérance aux défaillances** : Les services non concernés par une panne continuent à fonctionner, renforçant la robustesse de l'exécution ;
- **Interopérabilité cloud-native** : Le moteur est compatible avec Kubernetes, Docker, REST, facilitant l'intégration dans des pipelines DevOps.

Discussion

La composition incrémentale des services, telle qu'implémentée via les Grammaires Attribuées Gardées (GAG), présente plusieurs avantages notables dans le contexte des architectures orientées services. Tout d'abord, le caractère paresseux de l'évaluation permet de réduire significativement la charge computationnelle en ne recalculant que les sous-arbres affectés par une modification de la requête ou des données d'entrée. Cette granularité fine améliore la réactivité du système, particulièrement dans les scénarios où les données évoluent fréquemment ou de manière asynchrone.

En outre, l'utilisation d'une approche déclarative sépare clairement la spécification de la composition du code d'exécution, facilitant la maintenabilité et l'extensibilité du moteur de composition. La modularité introduite par la définition de services paresseux permet également de réutiliser des composants au sein de différents pipelines de traitement, contribuant à la cohérence et à la réduction de la duplication de logique métier.

Cependant, cette approche présente également des limites. Le surcoût initial lié à l'instanciation du moteur GAG et à la construction de l'arbre de dépendances peut entraîner une latence plus élevée pour les petites requêtes ponctuelles, par rapport à une composition statique optimisée. De même, la gestion de l'état intermédiaire et la persistance des attributs gardés nécessitent un accompagnement sophistiqué (caches, invalidation fine), sous peine de voir la complexité mémoire s'envoler dans des traitements à large échelle.

Enfin, bien que la GAG offre un formalisme puissant pour exprimer la composition incrémentale, son adoption reste encore limitée dans l'industrie. En résumé, la composition incrémentale via GAG représente une avancée prometteuse pour les systèmes orientés services à forte dynamique de données, mais sa maturité devra être consolidée par des optimisations du moteur, une meilleure prise en charge de l'environnement distribué et des efforts de standardisation pour convaincre les praticiens de l'adopter.

II.5. Conclusion

Le panorama des travaux présentés dans ce chapitre met en évidence l'évolution des architectures logicielles distribuées, depuis les approches monolithiques et SOA vers les microservices et l'émergence de la composition incrémentale. Nous avons montré que, face aux limites des compositions statiques et dynamiques classiques, les Grammaires Attribuées Gardées (GAG) offrent un modèle déclaratif et paresseux particulièrement adapté aux systèmes à forte variabilité de données. L'étude des principaux modèles a permis de dégager les points forts et faibles de chaque approche, la GAG se démarque par sa séparation claire entre spécification et exécution, ainsi que par sa capacité à ne recalculer que les portions réellement impactées. Cependant, ces bénéfices s'accompagnent de défis non négligeables : complexité initiale de mise en œuvre, coûts de synchronisation dans des environnements distribués, et maturité encore limitée côté industrialisation et outils de support. Ce bilan ouvre naturellement sur le chapitre suivant : la conception et l'évaluation expérimentale d'un benchmark basé sur la transformée de Fourier, qui permettra de quantifier précisément l'impact de la composition incrémentale via GAG sur des cas d'usage concrets.

III

CHAPITRE

UN BENCHMARK D'ÉVALUATION DE LA COMPOSITION INCRÉMENTALE DES SERVICES

SOMMAIRE

III.1 - Introduction	52
III.2 - Mise en place de la solution	53
III.3 - Conditions expérimentales	61
III.4 - Résultats et observations	63
III.5 - Conclusion	68

III.1. Introduction

À l'issue d'une revue approfondie de la littérature sur la composition incrémentale présentée dans le chapitre précédent, il apparaît clairement que, si les moteurs basés sur les Grammaires Attribuées Gardées (GAG) ont démontré leur faisabilité — *notamment à travers un prototype de tri récursif* [48] —, leur impact concret sur la performance reste à quantifier.

Dans ce chapitre, nous proposons un protocole de *benchmarking* basé sur la transformée de Fourier Rapide (FFT). Ce choix se justifie par sa large utilisation dans des domaines tels que le traitement du signal ou les communications numériques, ainsi que par sa complexité algorithmique bien connue, en $O(n \log n)$. La FFT suit une structure naturelle de type *Diviser-Récursion-Combiner*, qui en fait un cas d'étude idéal pour évaluer la composition incrémentale de services.

Chaque phase de cet algorithme peut être modélisée comme un microservice indépendant, déployé dans un conteneur Docker et orchestré dynamiquement par

Kubernetes. L'exécution est pilotée par un moteur basé sur les Grammaires Attribuées Gardées (GAG) ou, plus généralement par tout algorithme récursif F suivant le schéma : « diviser » $\rightarrow$ « appels récursifs sur F » $\rightarrow$ « combinaison ». Ce paradigme se traduit concrètement par une exécution distribuée : « appel du service de division » $\rightarrow$ « appels multiples de services récursifs sur F » $\rightarrow$ « appel du service de combinaison ». Cette modélisation permet de tirer pleinement parti des capacités d'exécution paresseuse et parallèle de la composition incrémentale.

L'objectif de ce chapitre est double : d'une part, établir une méthodologie rigoureuse — *de la mise en place des services FFT à la collecte des métriques de parallélisme, robustesse et scalabilité* —, et d'autre part, comparer de manière empirique l'approche incrémentale à un déploiement non incrémental. Grâce à ce benchmark reproductible, nous visons à quantifier précisément les gains de performance offerts par la composition incrémentale et à dégager des recommandations pratiques pour l'exploitation de ces moteurs sur des infrastructures hétérogènes.

III.2. Mise en place de la solution

Dans cette section, nous présentons la démarche adoptée pour modéliser et déployer une version distribuée et paresseuse de l'algorithme FFT en nous appuyant sur les GAG. L'objectif est de mettre en œuvre une composition incrémentale fondée sur le paradigme classique *Diviser – Récursion – Combiner*, où chaque étape du calcul est encapsulée dans un service distinct.

Bien que des algorithmes FFT bien établis permettent d'obtenir une complexité en temps de l'ordre de $O(n \log n)$, ils se heurtent à une difficulté majeure dans les contextes applicatifs modernes : *la gestion de grandes quantités de données*. Pour pallier ce problème, plusieurs implémentations parallèles ont été proposées, afin de tirer parti du parallélisme matériel. Cependant, ces approches restent souvent réservées aux utilisateurs disposant de ressources matérielles spécifiques (notamment des cartes graphiques hautes performances).

Dans cette optique, les implémentations distribuées de la FFT se révèlent particulièrement adaptées : elles permettent de répartir les calculs sur plusieurs nœuds interconnectés et de mieux exploiter les infrastructures modernes de calcul, comme les clusters cloud. En adoptant une approche basée sur les GAG, nous visons à rendre la FFT distribuée à la fois plus modulaire, plus flexible, et surtout plus accessible — même à des utilisateurs ne disposant que d'infrastructures standards.

III.2.1. Approche méthodologique

L'algorithme FFT suit une approche classique *DPR*. Un vecteur d'entrée x de taille $N = 2^k$ est divisé en deux sous-vecteurs : l'un regroupant les éléments d'indice pair, l'autre ceux d'indice impair. Chacun de ces sous-vecteurs est ensuite traité récursivement par la même fonction, jusqu'à atteindre le cas de base, où le vecteur ne contient qu'un ou deux éléments. À ce stade, le calcul peut être réalisé directement à l'aide de la formule de la DFT.

Dans le cas général, chaque appel récursif renvoie la transformée (FFT) de son sous-vecteur. Ces deux résultats intermédiaires sont ensuite combinés selon la formule bien connue :

$$X[k] = E[k] + W_N^k \cdot O[k] \quad \text{et} \quad X[k + N/2] = E[k] - W_N^k \cdot O[k]$$

où E et O représentent respectivement les FFT du sous-vecteur pair et impair, et W_N^k la k -ième racine N -ième de l'unité. Ce découpage naturel rend l'algorithme hautement parallélisable, les sous-transformées pouvant être calculées indépendamment.

notre approche exploite cette propriété en encapsulant chaque appel récursif dans un service autonome. La fonction FFT est ainsi implémentée comme un microservice conteneurisé, déployé au sein d'un cluster Kubernetes. L'orchestrateur est chargé d'assigner dynamiquement les appels récursifs aux nœuds disponibles, en fonction de la charge du système. Une API REST permet la communication entre les différentes instances, assurant la sérialisation des données et la coordination du calcul distribué.

Cependant, cette architecture classique introduit une dépendance forte entre les services : *la combinaison finale des résultats requiert que les deux sous-transformées soient disponibles simultanément*. Une panne sur l'un des deux services bloque donc le processus, ce qui limite la résilience globale de l'application.

Pour atténuer cette contrainte, nous proposons une variante incrémentale et paresseuse de la FFT distribuée, fondée sur les GAG. L'idée est de diviser le signal initial en k blocs élémentaires, chacune étant confiée à un service FFT. À chaque appel récursif, ces blocs peuvent être divisés localement en sous-blocs (paires et impaires), puis traités indépendamment.

Une fois les résultats disponibles, la combinaison peut s'effectuer dès que deux blocs transformés sont prêts, sans attendre l'ensemble du signal. Cela permet à l'algorithme de produire des résultats partiels au fil de l'eau. En cas de panne d'un service, le système peut continuer à traiter les blocs disponibles et combiner progressivement les résultats, réduisant ainsi les effets de blocage.

Cette stratégie améliore sensiblement la tolérance aux pannes, au prix d'un léger surcoût computationnel, tout en s'inscrivant naturellement dans une logique de composition incrémentale distribuée. Elle met à profit les capacités expressives des GAG pour orchestrer dynamiquement les étapes du calcul selon les dépendances effectives entre les blocs.

Pseudocode de la FFT paresseuse. Dans notre approche, le vecteur d'entrée est découpé en blocs élémentaires appelés *cellules*. Chaque cellule est traitée indépendamment par un service, qui effectue la transformation sur un petit sous-ensemble de données.

Les résultats partiels ainsi produits (sous-cellules combinées) sont ensuite combinés par des services intermédiaires selon la structure récursive de la FFT. Chaque service peut être déclenché dès que deux cellules transformées sont disponibles, permettant une exécution asynchrone, incrémentale et tolérante aux pannes. Le traitement se poursuit jusqu'à ce que le service racine obtienne le spectre complet.

Cette logique peut être exprimée par un pseudocode simplifié, illustrant la récursivité et la production partielle des résultats (Algo. 2) :

Algorithm 2: Algorithme FFT Paresseux

Input: Signal d'entrée x , nombre de blocs `blocSize`

Output: Résultat partiel ou complet de la FFT paresseuse

```

1 if  $\text{taille}(x) \leq \text{seuil}$  then
2   | appliquer BaseFFT( $x$ )
3 Découper  $x$  en blocSize blocs de taille égale
4 for chaque bloc do
5   | séparer les éléments d'indice pair et impair
6   | former deux listes : evenBlocs et oddBlocs
7  $E \leftarrow \text{FFT\_lazy}(\text{evenBlocs})$ 
8  $O \leftarrow \text{FFT\_lazy}(\text{oddBlocs})$ 
9 for chaque couple  $(E_k, O_k)$  disponible do
10  |  $X_k \leftarrow E_k + W^k \cdot O_k$ 
11  |  $X_{k+N/2} \leftarrow E_k - W^k \cdot O_k$ 
12  | if bloc  $X$  est complet et le service courant  $\neq$  racine then
13  |   | transmettre  $X$  au parent

```

Cette logique est ensuite traduite en règle GAG, dans laquelle chaque opération de l'algorithme devient une règle autonome, pouvant être exécutée indépen-

damment des autres. Grâce à l'abstraction offerte par le modèle, l'exploitation du parallélisme dans un environnement distribué devient transparente pour le développeur : le moteur GAG se charge de gérer l'expansion du graphe de composition, tandis que l'orchestrateur (Kubernetes) alloue dynamiquement les ressources en fonction de la charge, garantissant ainsi une exécution optimisée sans effort manuel supplémentaire.

Notre conception distingue deux types complémentaires de services :

- **Les services internes**, interprétés directement par le moteur GAG, correspondent aux symboles non terminaux de la grammaire. Ils définissent la structure logique du traitement à réaliser (décomposition, appels récursifs, combinaison, etc.) ;
- **Les services externes**, quant à eux, sont gérés par l'orchestrateur. Il s'agit des entités concrètes déployées dans le cluster (sous forme de pods), et que l'application peut invoquer dynamiquement via API REST.

Pour assurer un couplage cohérent entre ces deux couches (*logique et infrastructurelle*), chaque service interne est associé à une entrée explicite dans le fichier de spécification YAML. Ce fichier décrit le nom du service, ses entrées et sorties, ses gardes éventuelles, les sous-services enfants, ainsi que la correspondance entre les attributs.

Trois services fondamentaux émergent de cette modélisation :

- **MainFFT** : service racine qui initie le calcul ; il décompose le signal d'entrée en blocs, délègue le traitement au service récursif, puis combine les résultats obtenus ;
- **RecFFT** : service récursif qui applique la logique de division des blocs et d'appels imbriqués ;
- **CombineFFT** : service qui se charge de la combinaison de deux blocs.

Chaque règle GAG est associée à des attributs (entrées, sorties) et activée sous condition (garde). Ce formalisme permet de représenter explicitement les dépendances de données, et d'exécuter le calcul de manière progressive, distribuée et faiblement synchronisée, tout en garantissant la cohérence finale du résultat.

```
1 # Service principal : lance la composition FFT sur le signal
  complet
2 ---
3 kind: service
4 name: MainFFT
5 kubernetes: ${KUBE_NAME}
6 inputs:
7   - { name: in_signal, array: true, size: ${NUMBER_OF_BLOCKS} }
```

```
8 outputs:
9   - { name: spectrum }
10
11 # Service récursif : divise le signal en sous-blocs et les
   traite récursivement
12 ---
13 kind: service
14 name: RecFFT
15 kubename: ${KUBE_NAME}
16 inputs:
17   - { name: signal, array: true, size: ${NUMBER_OF_BLOCKS} }
18 outputs:
19   - { name: spectrum, array: true, size: ${NUMBER_OF_BLOCKS} }
20
21 # Service de combinaison : combine les parties paires et
   impaires après FFT
22 ---
23 kind: service
24 name: CombineFFT
25 kubename: ${KUBE_NAME}
26 inputs:
27   - { name: even, array: true, size: ${NUMBER_OF_BLOCKS} }
28   - { name: odd, array: true, size: ${NUMBER_OF_BLOCKS} }
29 outputs:
30   - { name: output }
```

Listing III.1 – *Declaration des Services*

Chaque service possède un champ `name` (listing III.1, ligne 4) qui correspond au nom du service interne, c'est-à-dire à la règle interprétée par le moteur GAG. Le champ `kubename` (listing III.1, ligne 5), quant à lui, représente le nom du service externe tel qu'il est connu de l'orchestrateur Kubernetes, et permet son invocation effective au moment du déploiement.

III.2.2. Architecture du prototype

Le prototype de notre solution, initialement mis en œuvre dans [48] lors de l'implémentation du tri incrémental, repose sur une chaîne de traitement en trois étapes principales (Figure 14) :

1. **Spécification de service** : un fichier YAML décrit la composition des services, incluant la déclaration de services externes et l'appel à des fonctions locales implémentées en Java. Ce langage de description permet de définir des services composites fondés sur les GAG ;
2. **Construction de l'image Docker** : l'*Image Builder* compile les spécifications YAML et le code Java pour produire la sémantique de service interpré-

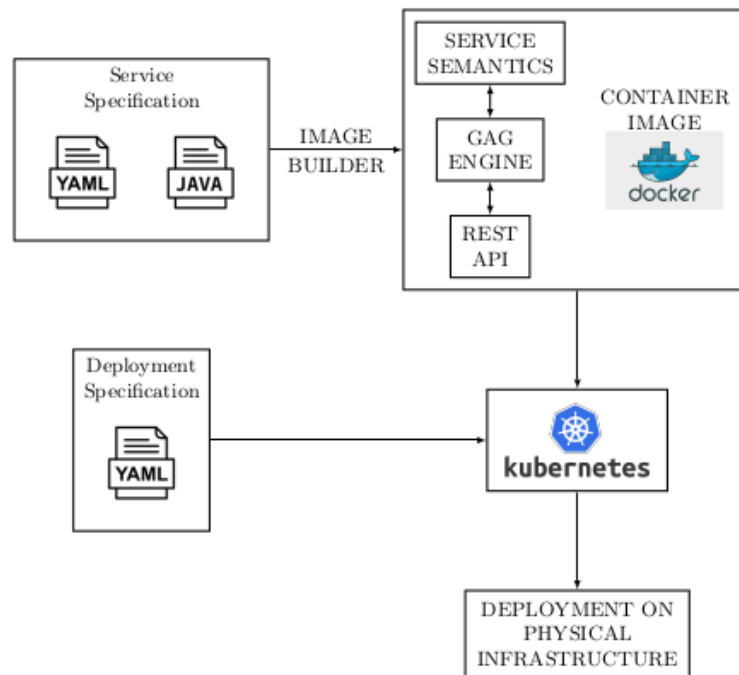


FIGURE 14 – Architecture du prototype [48]

table par le moteur GAG. Cette sémantique, le moteur GAG et une API REST sont empaquetés dans un conteneur Docker unique ;

3. **Orchestration Kubernetes :** à partir d'un second fichier YAML définit la spécification de déploiement, Kubernetes déploie alors chaque conteneur sur un cluster, gérant automatiquement l'allocation des ressources, la montée en charge et la tolérance aux pannes.

III.2.3. Processus de déploiement de l'algorithme

Production. La sémantique associée à chaque service est définie par la **règle de production** qui lui est liée. Lorsque le service n'est pas décomposable en plusieurs alternatives, il est nécessaire de lui associer une garde. Cette garde est généralement une fonction qui retourne toujours la valeur *true*, indiquant que la règle est toujours applicable pour résoudre le service. Elle joue un rôle formel pour satisfaire le schéma de la grammaire, même si elle ne conditionne pas réellement l'activation de la règle.

La section `functions` (listings III.2, ligne 8) permet de déclarer les *fonctions locales* utilisées par la règle, et chaque fonction peut être identifiée par un alias via le champ `label` (listings III.2, lignes 10, 12). Ces alias facilitent ensuite leur utilisation dans la section `actions` (listings III.2, ligne 13), qui décrit les règles

sémantiques de la production.

```

1 ---
2 kind: rule
3 name: Main_FFT
4 service: MainFFT
5 guard:
6   classPath: com.local.FourierFunc
7   method: checklength
8 functions:
9   - { classPath: com.local.FourierFunc, method: split,
10       label: split, multiOutput: true, outputSize: ${NUMBER_OF_BLOCKS} }
11   - { classPath: com.local.FourierFunc, method: combine_f,
12       label: combine }
13 actions:
14   - blocks = split(in_signal)
15   - result_blocks = RecFFT(blocks)
16   - half1 = CombineFFT(result_blocks[0], result_blocks[1])
17   - half2 = CombineFFT(result_blocks[2], result_blocks[3])
18   - spectrum = combine(half1, half2)

```

Listing III.2 – Production du service MainFFT

```

1 ---
2 kind: rule
3 name: Rec_FFT_remote
4 service: RecFFT
5 guard:
6   classPath: com.local.FourierFunc
7   method: checklength_in
8 functions:
9   - { classPath: com.local.FourierFunc, method: divide,
10       label: divide, multiOutput: true, outputSize: ${NUMBER_OF_BLOCKS} }
11 actions:
12   - part1, part2 = divide(signal)
13   - res1 = RecFFT(part1)
14   - res2 = RecFFT(part2)
15   - out1 = CombineFFT(res1[0], res1[1])
16   - out2 = CombineFFT(res1[2], res1[3])
17   - out3 = CombineFFT(res2[0], res2[1])
18   - out4 = CombineFFT(res2[2], res2[3])
19   - spectrum = [out1, out2, out3, out4]

```

Listing III.3 – Une des productions du service RecFFT

```

1 ---
2 kind: rule
3 name: Combine
4 service: CombineFFT

```

```
5 guard:
6   classPath: com.local.FourierFunc
7   method: combine_guard
8 functions:
9   - { classPath: com.local.FourierFunc, method: twiddle_combine,
10      label: twiddle_combine }
11 actions:
12   - output = twiddle_combine(even, odd)
```

Listing III.4 – Production du service *CombineFFT*

Le déploiement de l'algorithme s'appuie sur un environnement distribué simulé localement à l'aide de machines virtuelles. Pour cela, nous utilisons un dossier qui regroupe l'ensemble des ressources nécessaires à la création du cluster. Celui-ci est généré à l'aide des outils Vagrant[17] et VirtualBox[12]. Chaque machine virtuelle agit comme un nœud logique du cluster.

Compilation des services. Avant de pouvoir déployer les services, il est nécessaire de générer une image Docker pour chacun d'entre eux. Chaque image regroupe l'ensemble des éléments nécessaires à l'exécution autonome du service concerné. Cela inclut :

- le code compilé du moteur GAG, responsable de l'interprétation et de l'évaluation des règles de composition ;
- les fichiers de spécification des services, écrits en YAML, définissant la structure du service, ses attributs, ses dépendances et ses sous-services ;
- les règles de production, également spécifiées en YAML, décrivant les conditions de déclenchement (gardes) et les actions associées ;
- les fonctions locales implémentées en Java, utilisées pour effectuer des calculs ou manipulations internes spécifiques à chaque service.

L'ensemble de ces éléments est compilé et rassemblé à l'aide d'un outil de construction d'image interne. Ce dernier compile le code source, génère les binaires nécessaires, et assemble le tout dans une image Docker exécutable.

En principe, les fichiers de spécification et les règles pourraient être analysés à la compilation et traduits en structures internes prêtes à l'emploi. Toutefois, dans le cadre de ce prototype, cette analyse est volontairement déportée à l'exécution pour faciliter le développement et tester différentes configurations de manière souple. C'est donc le moteur GAG lui-même qui charge dynamiquement les règles au moment du lancement du conteneur.

Déploiement sur le cluster Kubernetes. Une fois les images Docker créées, leur déploiement est orchestré à l'aide d'un fichier de spécification Kubernetes,

rédigé au format YAML. Ce fichier référence les images construites et définit, pour chaque service :

- les ressources nécessaires (CPU, mémoire, nombre de répliques) ;
- les politiques de redémarrage, de tolérance aux pannes et d'équilibrage de charge ;
- les adresses ou ports d'exposition pour la communication REST.

L'exécution des commandes Kubernetes standard (telles que `kubectl apply`) suffit alors à déclencher le déploiement automatique des services sur les nœuds du cluster. Chaque image est instanciée sous forme de pod, et le moteur GAG contenu dans chaque conteneur pilote les appels de services à la volée, en fonction de l'état des données disponibles.

Ce processus garantit à la fois une gestion flexible du déploiement, une évolutivité naturelle via la déclaration du nombre de répliques, et une robustesse améliorée par l'isolation des services.

III.3. Conditions expérimentales

III.3.1. Environnement d'expérimentation

L'environnement d'expérimentation mis en place dans le cadre de ce travail vise à évaluer les performances de la composition incrémentale sur des services distribués, modélisés à l'aide de Grammaires Attribuées Gardées (GAG). Pour assurer un déploiement contrôlé, reproductible et conforme aux scénarios d'exécution distribuée, nous avons opté pour une infrastructure virtualisée localement, articulée autour de conteneurs Docker et d'un cluster Kubernetes.

Infrastructure de déploiement. Les expérimentations ont été réalisées sur une machine personnelle moyenne de gamme avec un processeur Intel core i7 (4 cœurs, 2 threads) et une mémoire vive (RAM) de 8Go.

Virtualisation et orchestration. L'infrastructure distribuée a été simulée localement à l'aide de deux outils principaux :

- *Vagrant* : utilisé pour l'automatisation de la configuration et du provisionnement des machines virtuelles ;
- *VirtualBox* : utilisé comme hyperviseur léger pour exécuter les VM localement.

Sur cette base, un cluster Kubernetes a été déployé manuellement, avec la configuration suivante :

- *1 nœud maître* : chargé de la gestion du plan de contrôle au sein du cluster ;

- 2 *nœuds workers* : utilisés pour héberger et exécuter les services conteneurisés.

Chaque machine virtuelle dispose de :

- 2 *vCPU* ;
- 2048 *Mo de RAM* pour le nœud maître ;
- 1536 *Mo de RAM* pour chaque nœud worker ;
- une adresse IP statique permettant la communication fluide entre les nœuds (consécutives à partir de 10.0.0.10).

III.3.2. Paramètres de test

Les tests expérimentaux menés dans ce travail visent à évaluer les performances de la solution proposée selon plusieurs critères : **modularité**, **robustesse**, **capacité de parallélisation** et **résilience**. L'objectif est de comparer le comportement d'un système fondé sur la composition incrémentale de services avec celui d'une implémentation centralisée plus classique de l'algorithme FFT.

Pour cela, différents paramètres de test ont été définis et modifiés de manière systématique afin d'observer leur influence sur le déroulement du calcul et sur les performances obtenues.

Taille du signal d'entrée. La taille du vecteur d'entrée soumis à la transformée de Fourier est un facteur clé dans l'analyse des performances. Les valeurs testées sont des puissances de 2, compatibles avec l'implémentation FFT récursive. Les tailles typiques testées vont de $N = 2^{10} = 1024$ à $N = 2^{17} = 131072$. Cette variation permet d'analyser la scalabilité du système et son comportement face à des volumes croissants de données.

Seuil de découpage (bloc minimal). Un seuil est défini pour contrôler la granularité des blocs de données à partir desquels un service local applique la FFT de manière directe (algorithme naïf). Plus ce seuil est bas, plus la profondeur de l'arbre d'appels récursifs sera grande. Cela permet d'étudier l'impact de la profondeur de récursion sur la performance globale et sur le niveau de parallélisme atteint.

Niveau de parallélisme. Pour évaluer la capacité du système à exploiter le parallélisme offert par l'architecture distribuée, nous avons fait varier le nombre de pods déployés simultanément dans le cluster. Chaque pod exécute un ou plusieurs services dérivés des règles GAG, correspondant à une sous-tâche du traitement (séparation, FFT locale, combinaison, etc.).

Les configurations testées vont de 1 à 5 pods. Cette variation permet d'évaluer les gains de performance obtenus par la distribution des traitements FFT sur plusieurs conteneurs isolés.

Contrairement à une augmentation du nombre de nœuds physiques ou virtuels, cette approche contrôle le nombre d'instances de traitement logiques, tout en gardant constante l'infrastructure (nombre de nœuds Kubernetes). Elle est particulièrement adaptée pour tester l'efficacité de la gestion des ressources par l'orchestrateur (scheduler Kubernetes) et observer l'impact du niveau de composition des services sur les performances globales.

III.4. Résultats et observations

III.4.1. Exécution distribuée

Nous présentons dans cette section les résultats obtenus lors de l'évaluation expérimentale du système basé sur la composition incrémentale de services pour le calcul distribué de la transformée de Fourier rapide (FFT). L'objectif principal est d'analyser l'impact du parallélisme sur le temps d'exécution global, en fonction de la taille du signal en entrée.

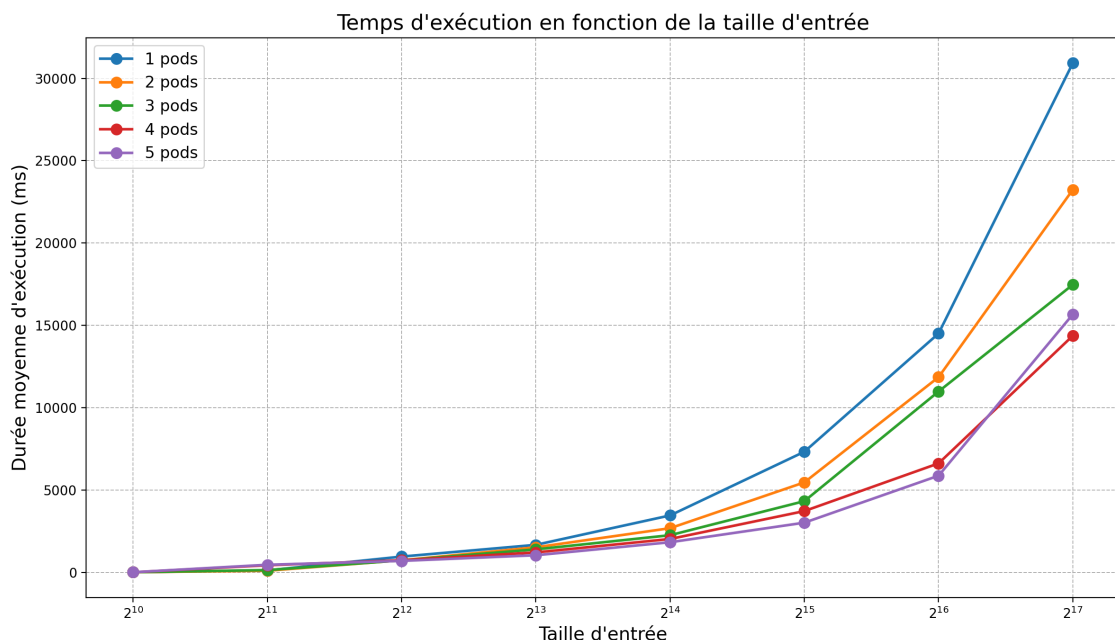


FIGURE 15 – Temps d'exécution moyen en fonction de la taille d'entrée pour différentes configurations de parallélisme

Analyse des performances. La Figure 15 illustre l'évolution du temps d'exécution moyen (en millisecondes) en fonction de la taille du signal d'entrée (en puissances de deux), pour des configurations comportant entre 1 et 5 pods actifs dans le cluster Kubernetes.

On observe les tendances suivantes :

- Pour les petites tailles d'entrée (2^{10} à 2^{13}), les temps d'exécution restent relativement proches quelle que soit la configuration du cluster (de 1 à 5 pods). Cela s'explique par deux facteurs principaux.

D'une part, la charge de calcul étant faible, le gain potentiel dû au parallélisme reste limité : la surcharge liée à la coordination et au déclenchement des services devient proportionnellement plus importante que le travail de transformation lui-même.

D'autre part, l'architecture du système repose sur une communication interservices via une API REST, utilisant le protocole HTTP. À chaque appel de service, les blocs de données doivent être sérialisés, transmis sous forme de requêtes, puis désérialisés à la réception. Ce mécanisme introduit un surcoût fixe en temps.

Lorsque les entrées sont de taille réduite, ce coût de sérialisation/désérialisation devient prépondérant par rapport au temps de calcul effectif. Il en résulte une latence importante et un amortissement inefficace du coût d'invocation de service, rendant l'exécution parallèle moins avantageuse dans ces cas ;

- À partir de 2^{14} , l'impact du nombre de pods devient significatif. L'utilisation de plusieurs pods permet une meilleure répartition des calculs FFT sur les sous-tableaux, réduisant ainsi le temps total de traitement ;
- Pour les très grandes tailles d'entrée (2^{16} à 2^{17}), l'ajout de pods supplémentaires continue de réduire le temps d'exécution, mais les gains deviennent de plus en plus faibles. Par exemple, le passage de 4 à 5 pods n'apporte qu'un bénéfice marginal.

Ce phénomène s'explique principalement par une saturation des ressources physiques de la machine hôte. Le cluster étant simulé localement à l'aide de machines virtuelles, les performances globales dépendent directement de la capacité du système hôte (CPU, mémoire disponible). Or, à partir d'un certain niveau de parallélisme, toutes les ressources CPU et/ou mémoire sont déjà sollicitées. Le système n'est alors plus en mesure de répartir efficacement la charge supplémentaire.

Par ailleurs, plus le nombre de pods augmente, plus la coordination entre services devient coûteuse : échanges de messages, appels HTTP, sérialisation/désérialisation. Lorsque ces surcoûts dépassent les gains liés à la parallélisation, l'efficacité globale du système stagne, voire diminue.

En résumé, le plateau de performance observé pour un grand nombre de pods reflète à la fois les limites matérielles de l'environnement d'exécution et le coût de coordination inhérent à l'architecture distribuée.

Observation sur la scalabilité. Ces résultats confirment que la composition incrémentale permet d'exploiter efficacement les capacités de parallélisme offertes par un cluster Kubernetes. L'architecture mise en place est capable de réduire sensiblement les temps de calcul, à condition que les ressources (pods) soient suffisamment nombreuses pour absorber la charge induite par la récursion de la FFT.

En résumé, le système démontre une bonne capacité d'adaptation aux charges croissantes tant que les ressources sont disponibles. Un dimensionnement adéquat du cluster, ainsi qu'une gestion équilibrée de la granularité des blocs, sont essentiels pour tirer pleinement profit de l'approche.

III.4.2. Comparaison approche incrémentale et non incrémentale

Les deux implémentations que nous comparons ici reposent sur le même formalisme de base : des grammaires attribuées gardées (GAG), utilisées pour modéliser la composition de services dirigée par les données. Elles visent toutes deux à orchestrer le calcul de la FFT dans un cadre distribué, via un ensemble de services réactifs.

La différence essentielle réside dans leur stratégie de traitement des données.

Dans la version **non incrémentale**, le signal d'entrée est considéré comme un tout : chaque service attend d'avoir reçu l'ensemble des résultats de ses sous-services avant de produire à son tour un bloc destiné à son parent. Le processus suit une logique synchrone et complète à chaque niveau, sans possibilité de progression partielle.

À l'inverse, l'approche **incrémentale** repose sur un découpage explicite des données en blocs. Chaque service peut produire un bloc intermédiaire dès qu'il a reçu deux blocs de ses sous-services. Cela permet une progression locale, incrémentale et asynchrone : les résultats remontent au fur et à mesure qu'ils deviennent disponibles, sans attendre la complétion totale des appels récursifs. Ce mécanisme permet d'exploiter plus finement le parallélisme et d'améliorer la réactivité globale du système.

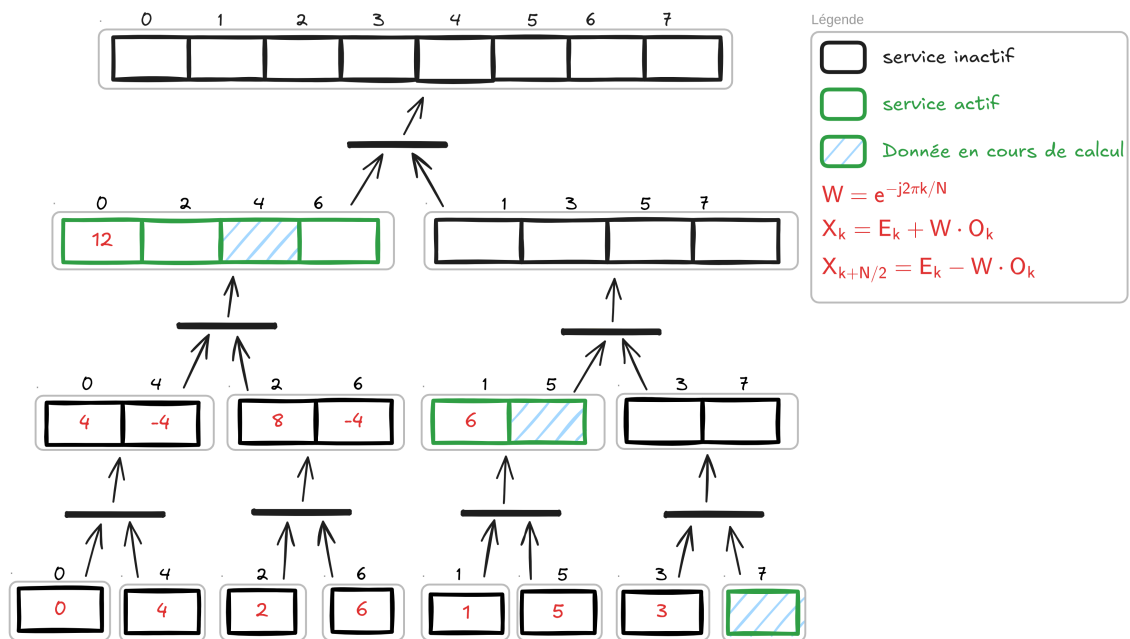


FIGURE 16 – Algorithme non incrémental

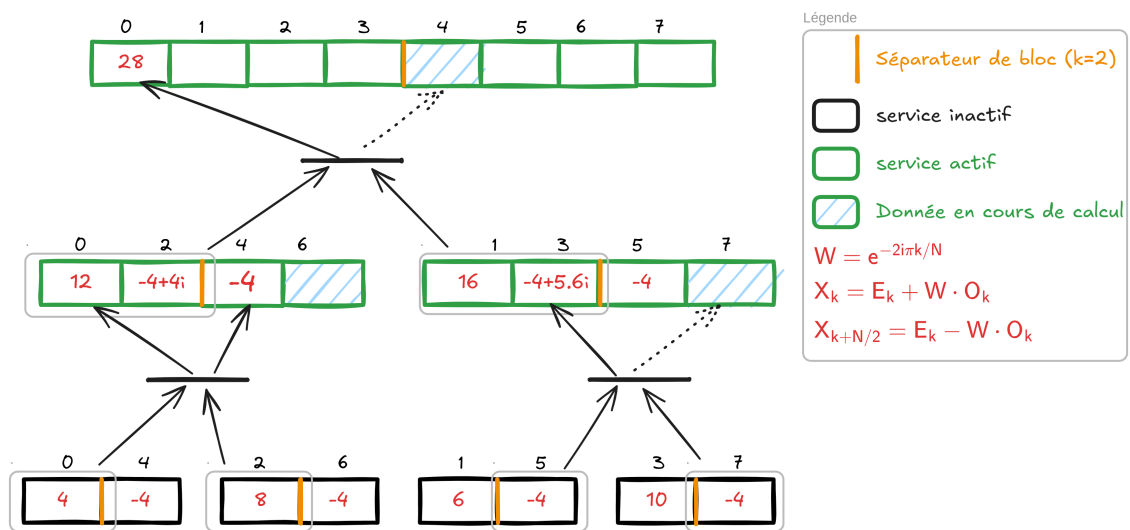


FIGURE 17 – Algorithme incrémental

Dans cette section, nous comparons ces deux variantes selon plusieurs critères expérimentaux, afin d'évaluer la pertinence de la composition incrémentale.

Les figures 16 et 17 illustrent deux stratégies d'exécution de l'algorithme de la transformée de Fourier rapide (FFT) : une version classique (non incrémentale) et une version incrémentale paresseuse.

La comparaison des différentes approches s'articule autour de trois axes : le *parallélisme*, la *robustesse* et la *latence*.

Parallélisme. Dans l'approche *non incrémentale* (Figure 16), le flux d'exécution est rigide et fortement centralisé. Sur tout chemin partant de la racine vers les feuilles, on observe qu'au plus un seul service est actif à un instant donné. Cette exécution suit une structure de type appel récursif, où chaque fonction attend les résultats de ses appels enfants avant de poursuivre. Cette séquentialisation limite fortement les opportunités de parallélisme naturel, sauf à introduire manuellement une orchestration spécifique.

À l'inverse, dans l'approche *incrémentale* (Figure 17), l'exécution suit une chaîne de production distribuée. Chaque sous-service peut traiter un fragment de données de manière autonome, puis transmettre un résultat partiel à son parent dès que possible. Cela permet l'activation concurrente de plusieurs services, favorisant ainsi une exécution parallèle fine, contrôlée par les dépendances de données. Cette propriété est inhérente à la sémantique paresseuse et aux mécanismes de synchronisation implicites induits par les GAG.

Robustesse. Sur le plan de la résilience, l'approche incrémentale présente également des avantages significatifs. Dans l'approche non incrémentale (Figure 16), un service traite une portion entière de l'entrée et produit son résultat de manière monolithique. Si ce service échoue en cours d'exécution, l'ensemble des données confiées est perdu, et le calcul doit redémarrer depuis le début ou un point de reprise explicitement programmé.

Dans l'approche incrémentale, au contraire, les données sont traitées par **blocs**. Lorsqu'un service échoue, il se peut qu'il ait déjà fourni une partie de ses résultats à son parent. On ne perd alors qu'une portion limitée du travail, ce qui rend le système plus robuste face aux défaillances. Cette **tolérance partielle** aux pannes est une propriété précieuse dans des environnements distribués et potentiellement instables.

Latence de résultat. En mode incrémental, dès qu'un fragment de données est prêt, il peut être immédiatement transmis au niveau supérieur, permettant une réduction de la latence. L'approche non incrémentale, elle, ne produit aucun résultat tant que l'ensemble du traitement n'est pas achevé, ce qui allonge le temps de réponse dans les systèmes interactifs.

Ces deux figures permettent de visualiser clairement les différences fondamentales entre les deux approches. Le tableau IV résume les principales différences entre les deux approches selon les trois axes étudiés.

TABLE IV – *Comparaison entre l'approche non incrémentale et l'approche incrémentale*

Critère	Approche non incrémentale	Approche incrémentale
Parallélisme	Flux d'exécution séquentiel et centralisé. Un seul service actif par chemin. Peu de parallélisme naturel.	Flux distribué sous forme de chaîne de production. Plusieurs services peuvent travailler en parallèle grâce à la distribution des dépendances.
Robustesse	Échec d'un service égale perte complète du travail sur la portion traitée. Redémarrage global souvent nécessaire.	Tolérance partielle aux pannes : seules les parties non transmises sont perdues. Reprise locale facilitée.
Latence des résultats	Les résultats sont produits uniquement à la fin du traitement complet.	Résultats partiels transmis dès qu'ils sont disponibles, réduisant la latence globale.

L'approche incrémentale, en s'appuyant sur une exécution paresseuse distribuée, offre des avantages majeurs en termes de parallélisme, robustesse et flexibilité, au prix d'une complexité accrue de coordination et d'une gestion fine des dépendances. Ces caractéristiques en font un candidat idéal pour les systèmes modernes orientés services.

III.5. Conclusion

Dans ce chapitre, nous avons présenté une expérimentation visant à évaluer la pertinence et l'efficacité de la composition incrémentale des services dans un contexte distribué, à travers l'implémentation d'un algorithme paresseux de la transformée de Fourier Rapide (FFT). Ce chapitre a démontré la faisabilité de la composition incrémentale basée sur les GAG dans un contexte réel d'algorithme intensif (FFT). Il a permis de valider expérimentalement la réduction de latence, l'augmentation du parallélisme et la flexibilité de l'approche par rapport à une exécution classique. Ces résultats renforcent l'intérêt d'une orchestration pilotée paresseusement, et ouvrent des perspectives d'amélioration du moteur GAG et d'élargissement à d'autres classes d'algorithmes.

CONCLUSION GÉNÉRALE

SOMMAIRE

Problématique étudiée et choix méthodologiques	69
Analyse critique des résultats obtenus	70
Quelques perspectives	70

Ce mémoire présente une recherche innovante à l'intersection de deux domaines informatiques distincts : les Grammaires Attribuées Gardées (GAG) comme paradigme de composition incrémentale des services, et la transformée de Fourier rapide (FFT) comme cas d'usage benchmark pour l'évaluation des performances.

Problématique étudiée et choix méthodologiques

La question de recherche principale porte sur l'évaluation de l'efficacité de la composition incrémentale dans un contexte de calcul intensif, dans quelle mesure les services paresseux peuvent-ils constituer une alternative viable aux approches traditionnelles pour l'implémentation d'algorithmes computationnellement exigeants, tout en préservant les avantages de la composition incrémentale ? À cet effet, nous avons d'abord modélisé un algorithme de FFT paresseux au moyen de GAG, permettant ainsi de dissocier la spécification déclarative (règles de productions) de l'exécution (fonctions locales). Ensuite, ces services ont été déployés dans un environnement conteneurisé (Docker) orchestré par Kubernetes sur un cluster. Enfin, nous avons élaboré un protocole expérimental mesurant la latence en fonction de la taille des transformées afin d'analyser l'impact de l'incrémental sur les performances.

Analyse critique des résultats obtenus

Les résultats expérimentaux révèlent que, d'une part, la latence reste maîtrisée pour des FFT de grande taille du fait que l'incrémentation ne concerne que les portions modifiées, et d'autre part, la scalabilité s'améliore grâce à la distribution fine des tâches FFT sur plusieurs nœuds Kubernetes sans recomposition globale. Par ailleurs, l'approche paresseuse limite l'utilisation mémoire en évitant un recalcul exhaustif, ce qui permet de rivaliser avec des compositions statiques optimisées. Toutefois, il convient de noter que pour des transformées de très petite taille, l'overhead initial du moteur GAG peut surpasser les gains apportés, et que la performance dépend étroitement de la granularité choisie dans les GAG, rendant critique un paramétrage adapté.

Quelques perspectives

Ce travail propose une approche certes prometteuse mais pas parfaite, c'est pourquoi plusieurs perspectives de recherche se posent :

- Étendre l'expérimentation à des infrastructures haut de gamme (bare-metal ou instances cloud) pour valider la scalabilité et la robustesse en conditions réelles ;
- Assurer la récupération automatique des données perdues en cas de panne de service ;
- Exploiter la parallélisation de l'opération de combinaison en l'exprimant sous forme matricielle pour mettre en lumière la pertinence des services paresseux sur des modèles de calcul parallèle ;
- Développer un module de supervision adaptative capable d'ajuster dynamiquement la granularité d'incrémentation en fonction de la charge des nœuds, afin de maximiser l'utilisation des ressources et d'optimiser le coût opérationnel ;
- Étendre l'usage de la composition incrémentale à des algorithmes non nécessairement de type DPR, mais qui présentent des structures de dépendances explicites entre leurs composants.

BIBLIOGRAPHIE

- [1] UDDI version 3.0.2. Technical report, 2004. Technical report.
- [2] Daniel Austin, A Barbir, C Ferris, and S Garg. Web services architecture requirements, w3c working group note. 11 february 2004. *Online at : <http://www.w3.org/TR/wsa-reqs/#id2604831>*, 2004.
- [3] Uchechukwu Awada. Application-container orchestration tools and platform-as-a-service clouds : A survey. *International Journal of Advanced Computer Science and Applications*, 2018.
- [4] Eric Badouel, Loïc Hélouët, Georges-Edouard Kouamou, Christophe Morvan, and Nsaibirni Robert Fondze Jr. Active workspaces : distributed collaborative systems based on guarded attribute grammars. *ACM SIGAPP Applied Computing Review*, 15(3) :6–34, 2015.
- [5] Arindam Banerji, Claudio Bartolini, Dorothea Beringer, Venkatesh Chopella, Kannan Govindarajan, Alan Karp, Harumi Kuno, Mike Lemon, Gregory Poggossiants, Shamik Sharma, et al. Web services conversation language (wscl) 1.0 w3c note. *World Wide Web Consortium*, 2002.
- [6] David Bernstein. Containers and cloud : From lxc to docker to kubernetes. *IEEE Cloud Computing*, 1(3) :81–84, 2014.
- [7] David Booth. Web services architecture. *W3C Note : <https://www.w3.org/TR/ws-arch/>*, 2004.
- [8] Thanh Bui. Analysis of docker security. *arXiv preprint arXiv :1501.02967*, 2015.
- [9] Kishore Channabasavaiah, Kerrie Holley, and Edward Tuggle. Migrating to a service-oriented architecture. *IBM DeveloperWorks*, 16 :727–728, 2003.

- [10] JiuJun Cheng, Cong Liu, MengChu Zhou, Qingtian Zeng, and Antti Ylä-Jääski. Automatic composition of semantic web services based on fuzzy predicate petri nets. *IEEE Transactions on Automation Science and Engineering*, 12(2) :680–689, 2014.
- [11] James W Cooley and John W Tukey. An algorithm for the machine calculation of complex fourier series. *Mathematics of computation*, 19(90) :297–301, 1965.
- [12] Oracle Corporation. Oracle virtualbox documentation. <https://www.virtualbox.org/wiki/Documentation>, 2024. Consulté le 3 janvier 2025.
- [13] AFAHOUNKO Danny. Virtualisation. 2013.
- [14] Docker Inc. Docker overview. <https://docs.docker.com/get-started/docker-overview/>, 2025. Consulté le 24 mai 2025.
- [15] Nicola Dragoni, Saverio Giallorenzo, Alberto Lluch Lafuente, Manuel Mazara, Fabrizio Montesi, Ruslan Mustafin, and Larisa Safina. Microservices : yesterday, today, and tomorrow. *Present and ulterior software engineering*, pages 195–216, 2017.
- [16] Roy Thomas Fielding. *Architectural styles and the design of network-based software architectures*. University of California, Irvine, 2000.
- [17] HashiCorp. Vagrant documentation. <https://developer.hashicorp.com/vagrant/docs>, 2025. Consulté le 3 janvier 2025.
- [18] IBM. SOA vs. microservices. <https://www.ibm.com/think/topics/soa-vs-microservices>, 2025. Consulté le 21 juin 2025.
- [19] Kasun Indrasiri and Danesh Kuruppu. *gRPC : up and running : building cloud native applications with Go and Java for Docker and Kubernetes*. O’Reilly Media, 2020.
- [20] Lvar Jacobson and James Rumbaugh Grady Booch. The unified modeling language reference manual. 2021.
- [21] Sanath Jayasena, Milinda Fernando, Tharindu Rusira, Chalitha Perera, and Chamara Philips. Auto-tuning the java virtual machine. In *2015 IEEE International Parallel and Distributed Processing Symposium Workshop*, pages 1261–1270. IEEE, 2015.

- [22] Eric Jonas, Johann Schleier-Smith, Vikram Sreekanti, Chia-Che Tsai, Anurag Khandelwal, Qifan Pu, Vaishaal Shankar, Joao Carreira, Karl Krauth, Neeraja Yadwadkar, et al. Cloud programming simplified : A berkeley view on serverless computing. *arXiv preprint arXiv :1902.03383*, 2019.
- [23] Nicolai M Josuttis. *SOA in practice : the art of distributed system design*. "O'Reilly Media, Inc.", 2007.
- [24] Ann Mary Joy. Performance comparison between linux containers and virtual machines. In *2015 international conference on advances in computer engineering and applications*, pages 342–346. IEEE, 2015.
- [25] Matjaz B Juric, Benny Mathew, and Poornachandra G Sarang. *Business process execution language for web services : an architect and developer's guide to orchestrating web services using BPEL4WS*. Packt Publishing Ltd, 2006.
- [26] Paul Klint, Ralf Lämmel, and Chris Verhoef. Toward an engineering discipline for grammarware. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 14(3) :331–380, 2005.
- [27] Heather Kreger et al. Web services conceptual architecture (wsca 1.0). *IBM software group*, 5(1) :6–7, 2001.
- [28] Kubernetes. Documentation kubernetes. <https://kubernetes.io/fr/docs/home/>, 2025. Consulté le 24 mai 2025.
- [29] Sandeep Kumar and Ravi Bhushan Mishra. A framework towards semantic web service composition based on multi-agent system. *International Journal of Information Technology and Web Engineering (IJITWE)*, 3(4) :59–81, 2008.
- [30] Frank Leymann et al. Web services flow language (wsfl 1.0). 2001.
- [31] Di Liu and Libin Zhao. The research and implementation of cloud computing platform based on docker. In *2014 11th international computer conference on wavelet active media technology and information processing (ICCWAMTIP)*, pages 475–478. IEEE, 2014.
- [32] Roberto Lucchi and Manuel Mazzara. A pi-calculus based semantics for ws-bpel. *The Journal of logic and algebraic programming*, 70(1) :96–118, 2007.
- [33] Peter Mell, Tim Grance, et al. The nist definition of cloud computing. 2011.

- [34] Dirk Merkel et al. Docker : lightweight linux containers for consistent development and deployment. *Linux j*, 239(2) :2, 2014.
- [35] Alexandru Moga, Thanikesavan Sivanthi, and Carsten Franke. Os-level virtualization for industrial automation systems : Are we there yet? In *Proceedings of the 31st Annual ACM Symposium on Applied Computing*, pages 1838–1843, 2016.
- [36] Sam Newman. *Building microservices : designing fine-grained systems*. "O'Reilly Media, Inc.", 2021.
- [37] Mike P Papazoglou. Service-oriented computing : Concepts, characteristics and directions. In *Proceedings of the Fourth International Conference on Web Information Systems Engineering, 2003. WISE 2003.*, pages 3–12. IEEE, 2003.
- [38] Mike P Papazoglou and Willem-Jan Van Den Heuvel. Service oriented architectures : approaches, technologies and research issues. *The VLDB journal*, 16 :389–415, 2007.
- [39] Joachim Peer. A pddl based tool for automatic web service composition. In *International Workshop on Principles and Practice of Semantic Web Reasoning*, pages 149–163. Springer, 2004.
- [40] Chris Peltz. Web services orchestration and choreography. *Computer*, 36(10) :46–52, 2003.
- [41] Aaqib Rashid and Amit Chaturvedi. Cloud computing characteristics and services : a brief review. *International Journal of Computer Sciences and Engineering*, 7(2) :421–426, 2019.
- [42] Abdul Saboor, Mohd Fadzil Hassan, Rehan Akbar, Syed Nasir Mehmood Shah, Farrukh Hassan, Saeed Ahmed Magsi, and Muhammad Aadil Siddiqui. Containerized microservices orchestration and provisioning in cloud computing : A conceptual framework and future perspectives. *Applied Sciences*, 12(12), 2022.
- [43] Quan Z Sheng, Xiaoqiang Qiao, Athanasios V Vasilakos, Claudia Szabo, Scott Bourne, and Xiaofei Xu. Web services composition : A decade's overview. *Information Sciences*, 280 :218–238, 2014.
- [44] Randall Smith. *Docker orchestration*. Packt Publishing Ltd, 2017.

- [45] Anshu Soni and Virender Ranga. Api features individualizing of web services : Rest and soap. *International Journal of Innovative Technology and Exploring Engineering*, 8(9) :664–671, 2019.
- [46] Clemens Szyperski, Dominik Gruntz, and Stephan Murer. *Component software : beyond object-oriented programming*. Pearson Education, 2002.
- [47] Joskel Ngoufo Tagueu. *An approach to modeling distributed collaborative systems with active workspaces through extended guarded attribute grammars*. PhD thesis, University of Dschang, Dschang, Cameroon, 2024.
- [48] Joskel Ngoufo Tagueu, Adrián Puerto Aubel, and Maurice Tchoupe Tchendji. A service composition engine for incremental computation. In *African Conference on Research in Computer Science and Applied Mathematics*, pages 156–171. Springer, 2024.
- [49] Joskel Ngoufo Tagueu, Eric Badouel, Adrián Puerto Aubel, and Maurice Tchoupe Tchendji. Lazy services : A service oriented architecture based on incremental computations and commitments. 2021.
- [50] David A Taylor. Object-oriented technology : A manager’s guide, addision-wesely. *Reading*, 1990.
- [51] Maurice Tchoupe Tchendji and Joskel Ngoufo Tagueu. Guarded attribute grammars and publish/subscribe for implementing distributed collaborative business processes with high data availability. *Service Oriented Computing and Applications*, 15(3) :245–256, 2021.
- [52] Steve Vinoski. An overview of middleware. In *International Conference on Reliable Software Technologies*, pages 35–51. Springer, 2004.
- [53] World Wide Web Consortium. Web service choreography interface (WSCI) 1.0. Technical report, W3C, Aug. 2002. Consulté le 21 juin 2025.